Coordination of Governed Agents Over Shared Context: Declarative Governance and Lyapunov-Based Finality

Jean-Baptiste Dézard*
Deal ex Machina SAS
jeanbapt@dealexmachina.com

March 2026

Abstract

The emergence of LLM-powered autonomous agents has exposed the limits of pipeline-based coordination: rigid DAGs cannot accommodate contradiction, evolving evidence, or formal convergence. We propose *declarative governance over shared immutable context*, a paradigm where autonomous agents reason over a common semantic graph, propose state transitions, and reach consensus through formal convergence mechanisms—without predefined execution sequences.

Six architectural contributions enable this: (1) an append-only event log and a *bitemporal* CRDT-inspired semantic graph providing shared, monotonic, temporally auditable context; (2) a *product lattice* $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ unifying governance levels (MASTER $\leq$ MITL $\leq$ YOLO) with a four-dimensional convergence rank into a single well-founded descent domain, with a deterministic *reduction kernel* that maps every proposal to Accept/Reject/Escalate through policy check, lattice admissibility, and mode routing; (3) per-dimension vector finality that prevents compensation attacks (where one dimension over-improves to mask failures in another), implemented as a conjunction of per-dimension thresholds (τ_d, ϵ_d) with gate refactoring (GA^d, GC^d per-dimension; GB, GD, GE global), providing stronger guarantees than scalar Lyapunov-based convergence tracking, with human-in-the-loop routing when convergence stalls; (4) Ed25519-signed finality certificates embedding policy-version hashes for cryptographic non-repudiation; (5) three-tier governance routing (deterministic rules, oversight agent, full LLM) with circuit-breaker fallback ensuring continuous operation without LLM availability; (6) a native Rust evaluation engine (compiled via napi-rs) executing all deterministic governance and convergence logic with zero LLM tokens, while TypeScript orchestrates I/O.

Unlike orchestration frameworks (AutoGen, CrewAI, LangGraph) that wire agents into chains or graphs, our system lets agents independently reason over shared state and propose actions evaluated by the reduction kernel against the product lattice. The lattice construction draws on classical information-flow lattice models [6] and termination theory [1]: every governance-approved transition must descend in $\mathcal{M}$, guaranteeing termination of each convergence epoch. The kernel's determinism enables replay, audit, and formal verification independently of LLM stochasticity.

Validation on two scenarios—M&A due diligence (Project Horizon) and multi-period financial consolidation with bitemporal restatement—demonstrates that $V(t)$ reaches 0.0000 at resolution epochs (the first empirical confirmation of full intra-epoch convergence), exhibits a sawtooth trajectory across evidence epochs as predicted by the perpetual lifecycle model, and produces domain-independent dynamics confirmed across both scenarios. Five-gate prevention of false finality is ablation-confirmed, and governance modes produce distinct coverage–autonomy positions. The architecture is validated by 141 Rust and 319 TypeScript tests.

*Source code: `https://github.com/DealExMachina/swarm-of-governed-agents`

Crucially, finality is not terminal: new evidence, regulatory amendments, or periodic review triggers re-open convergence over the same graph, producing a chain of certified checkpoints that models the perpetual lifecycle of regulated knowledge. The resulting architecture is designed for regulated environments where temporal auditability, policy traceability, ongoing monitoring, and cryptographic proof of every decision point are operational requirements.

1 Introduction

The rapid maturation of LLM-powered agents—capable of reasoning, planning, and tool use—has created a coordination problem that traditional workflow engines were not designed to solve. Modern agents do not merely execute predefined tasks; they interpret context, generate hypotheses, and produce claims that may contradict each other. This capability demands a fundamentally different coordination model.

Current approaches fall into two categories, both inadequate:

Pipeline orchestration (Airflow, Temporal, Prefect) assumes static task graphs executed in topological order. When agents produce contradictory outputs, the pipeline either silently overwrites earlier results or fails entirely. There is no mechanism for tracking disagreement, no formal convergence guarantee, and no governance trail explaining *why* a particular conclusion was reached.

Agent frameworks (AutoGen, CrewAI, LangGraph) represent progress: they enable multi-agent conversations, role-based delegation, and tool use. However, they still wire agents into predefined topologies—sequential chains, hierarchical delegation, or cyclic graphs with hardcoded routing. Coordination logic is embedded in code, not governed by policy. There is no shared immutable context, no formal finality mechanism, and no fine-grained access control.

We propose a third path: *declarative governance over shared context*. Rather than sequencing agents, the system maintains:

1. **Shared bitemporal context:** An append-only event log and a CRDT-inspired semantic graph with dual temporality (valid time and transaction time), where confidence scores ratchet upward, contradictions are tracked explicitly, and no state is ever deleted—only superseded. Bitemporal queries enable point-in-time reconstruction for regulatory audit.

2. **Product lattice and reduction kernel:** A product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ unifies a governance-level lattice $\mathcal{L}$ (MASTER $\leq$ MITL $\leq$ YOLO, ordered by permissiveness) with a four-dimensional convergence rank $\mathcal{A}$, forming a well-founded descent domain [4]. A deterministic *reduction kernel* evaluates every proposal against $\mathcal{M}$: policy rules check transition admissibility, the lattice verifies that the proposed state dominates the current state, and mode routing determines the verdict (Accept, Reject, or Escalate). This construction draws on classical lattice-based security models [2, 6] and rewriting-system termination theory [1]: every approved transition descends in $\mathcal{M}$, guaranteeing epoch termination. XACML combining algorithms [16] resolve multi-policy conflicts. Every decision produces an immutable, policy-version-stamped audit record.

3. **Formal convergence with five gates and perpetual lifecycle:** A Lyapunov disagreement function $V(t)$ tracks distance to targets across four weighted dimensions. Monotonicity gates, evidence coverage, oscillation detection, quiescence, and minimum-content gates provide layered guarantees against premature finality. When autonomous convergence stalls, the system routes to structured human review with full context. Finality is certified with Ed25519-signed JWS certificates—but finality is not terminal: new evidence, regulatory changes, or peri-

odic review triggers re-open convergence over the same graph, producing a chain of certified checkpoints that models the perpetual lifecycle of regulated knowledge.

4. **Three-tier governance routing:** Deterministic rule evaluation (zero LLM tokens), lightweight oversight agent, and full LLM-backed governance agent, with circuit-breaker fallback ensuring continuous operation without LLM availability.

5. **Fine-grained access governance:** OpenFGA enforces who can read documents, propose transitions, approve decisions, and modify policies.

This paper presents the design, formal properties, and experimental validation of this system, with particular attention to its fitness for regulated environments.

1.1 Contribution and Scope

The paper makes the following concrete contributions:

- An architectural paradigm—declarative governance over shared immutable context—that replaces predefined agent topologies with policy-driven coordination over a common semantic graph.
- A product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ that unifies governance admissibility and convergence rank into a single well-founded descent domain, with a deterministic reduction kernel guaranteeing epoch termination (Theorem 1).
- A formal convergence mechanism based on a Lyapunov disagreement function with five independent gates (monotonicity, evidence coverage, oscillation detection, quiescence, minimum content) that provides layered guarantees against premature finality.
- A perpetual finality lifecycle in which certified checkpoints—not terminal decisions—model the ongoing nature of regulated knowledge, with Ed25519-signed certificates binding each decision to the exact policy version that produced it.
- A three-tier governance routing architecture (deterministic rules, oversight agent, full LLM) with circuit-breaker fallback, ensuring continuous governance without LLM availability.
- Validation on two scenarios—M&A due diligence and financial consolidation with dual temporality—with 121 Rust and 306 TypeScript tests, demonstrating convergence, contradiction resolution, temporal restatement handling, and structured human review.

For clarity, the following are explicit limits of the paper's claims:

What the paper does not demonstrate.

- *Statistical generality.* The Project Horizon validation is a single scenario with synthetic documents; no confidence intervals, hypothesis tests, or distributional claims are supported.
- *Byzantine fault tolerance.* All agents are assumed cooperative. Adversarial agents that inject false claims, manipulate confidence scores, or game the governance protocol are not addressed. Integration with Byzantine-tolerant consensus machinery [3, 10] remains future work.

- *Scalability beyond proof of concept.* Validation covers 50 claims, 5 documents, and 7 agents. Coordination dynamics at production scale (thousands of claims, hundreds of agents) are untested.
- *Machine-checked convergence proofs.* Convergence guarantees are validated empirically through benchmarks and unit tests, not by formal verification tools or proof assistants.
- *Real LLM stochasticity.* Convergence benchmarks use synthetic trajectories. The oscillation detection and trajectory quality mechanisms are designed for real-world stochasticity but have not been validated against it.
- *Protocol self-modification.* Governance rules are declarative but static within a deployment. The system does not implement governed self-modification of its own coordination protocol, as studied by de la Chica Rodriguez and Vera Díaz [5].
- *Multi-scope finality.* Cross-scope governance, hierarchical finality (parent scope depending on child scopes), and inter-scope certificate chains are designed but not validated.

These boundaries are revisited in detail in Section 12.

2 Related Work

2.1 Pipeline Orchestration

Workflow engines (Apache Airflow, Temporal, Prefect) implement the DAG model: define a static graph, schedule tasks in topological order, apply retry logic on failure. This model is effective for deterministic ETL pipelines but breaks when agents produce conflicting outputs, when new evidence appears mid-execution, or when tasks are interdependent in non-linear ways. None provide formal finality guarantees or governance-as-coordination.

2.2 Agent Coordination Frameworks

Recent frameworks enable multi-agent collaboration:

AutoGen [22] introduces conversable agents with role-based chat patterns. Coordination is conversation-driven: agents exchange messages in predefined patterns (sequential, group chat, nested). However, coordination topology is hardcoded in Python, there is no shared persistent state between conversations, and no formal convergence mechanism.

CrewAI [14] organizes agents into crews with roles, goals, and tools. A hierarchical manager delegates tasks. Coordination follows a fixed process (sequential or hierarchical), with no mechanism for handling contradictions between agent outputs or tracking disagreement over time.

LangGraph [11] models agent coordination as state machines with explicit edges and conditional routing. This is closer to our approach—state is explicit and transitions are defined—but routing logic is embedded in code, not declarative policy. There is no immutable audit log, no CRDT semantics, and no formal convergence tracking.

All three frameworks assume that coordination topology can be predefined. Our system replaces topology with policy: agents reason independently over shared state, propose transitions, and governance rules determine what happens next.

2.3 Byzantine Consensus and Multi-Agent Coordination

The Byzantine consensus problem, formalized by Lamport, Shostak, and Pease [10], established that agreement among distributed processes is possible if and only if $n \geq 3f + 1$, where f is the number of Byzantine-faulty nodes. Practical Byzantine Fault Tolerance (PBFT) [3] made this feasible in partially synchronous systems with $O(n^2)$ message complexity. Ren et al. [19] surveyed consensus problems in multi-agent coordination, establishing the graph-theoretic foundations for state agreement under static and time-varying topologies. Zheng et al. [23] introduced confidence-weighted Byzantine fault tolerance (CP-WBFT), incorporating agent reliability into consensus—a direction relevant to LLM-based agents whose outputs carry inherent uncertainty. Mao et al. [13] extended the Byzantine Generals framework to practical multi-agent systems with the Imperfect Byzantine Generals Problem (IBGP), addressing local coordination without full global consensus.

Our system does not implement Byzantine consensus directly; agents are assumed cooperative. However, the semantic graph's monotonic constraints (confidence ratchets upward, contradictions are irreversible) provide partial robustness against downward manipulation, and the three-tier governance architecture limits the impact of any single agent's faulty output by requiring governance approval for all state transitions.

2.4 Self-Evolving Coordination Protocols

De la Chica Rodriguez and Vera Díaz [5] introduced Self-Evolving Coordination Protocols (SECP): coordination mechanisms that permit bounded, auditable self-modification of their own decision rules while preserving formal invariants. In a controlled feasibility study with six decision modules evaluating six Byzantine consensus proposals, they demonstrated that (i) non-scalar coordination rules can be synthesized by current AI models, (ii) a single governed modification increased proposal coverage by 50% while preserving declared invariants, and (iii) a systematic trade-off exists between protocol coverage (how many proposals are accepted) and evaluator autonomy (the capacity of individual modules to impose non-compensable objections).

Their work is complementary to ours in a precise sense. SECP addresses *protocol-level coordination*: how heterogeneous evaluator judgments are aggregated into accept/reject decisions, and how that aggregation rule can evolve. Our system addresses *state-level coordination*: how agents reason over shared context, how disagreement is tracked formally, and how convergence is certified. SECP's explicit open questions—multi-iteration convergence dynamics, formal invariant preservation under repeated modification, and audit trail infrastructure—are directly addressed by our Lyapunov-based convergence tracking, five-gate finality mechanism, and bitemporal audit graph. Conversely, their non-scalar coordination with non-compensable objection rights and Byzantine fault tolerance assumptions address gaps we acknowledge: our convergence metric is scalar, our governance rules are static within a deployment, and we do not handle adversarial agents.

A unified architecture combining SECP's governed protocol evolution with our formal convergence tracking would enable multi-iteration protocol modification where each iteration's effect on the Lyapunov disagreement function $V(t)$ is measured, gates prevent premature adoption of harmful modifications, and the certificate chain records the protocol version under which each decision was made.

2.5 Consensus and Convergence Theory

Olfati-Saber and Murray [17] established the Lyapunov consensus framework for multi-agent systems, proving that monotonically decreasing disagreement functions guarantee convergence under

mild assumptions. We adapt this to knowledge-work coordination, defining a weighted disagreement function over four semantic dimensions. The classical theory [12] provides the mathematical foundation: if a scalar function $V \geq 0$ is strictly decreasing along system trajectories and $V = 0$ only at the equilibrium, the system converges.

Ruan et al. [20] introduced the Aegean consensus protocol for multi-agent reasoning, which detects incremental quorum convergence across concurrent agent executions with provable refinement monotonicity—solution quality is guaranteed non-decreasing. We adapt their monotonicity principle to our finality evaluator: Gate A requires β consecutive non-decreasing rounds before declaring convergence.

2.6 CRDT Semantics and Monotonic State

Laddad et al. [9] demonstrated CRDT monotonic merge operations (CodeCRDT) that prevent state regression in distributed systems. Our semantic graph adopts this principle: confidence scores ratchet upward only, contradictions once marked are irreversible, and nodes are staled rather than deleted.

2.7 Stigmergic Coordination

Dorigo, Bonabeau, and Theraulaz [7] formalized stigmergy as a distributed communication paradigm: agents coordinate indirectly through local modifications to their environment (pheromone trails in ant colonies) rather than through direct message passing. We apply this principle to convergence-driven agent routing: each agent's activation is gated by the per-dimension convergence pressure of the shared semantic graph, causing agents to concentrate on bottleneck dimensions without centralized scheduling.

2.8 Bitemporal Data Models

Snodgrass [21] established the bitemporal data model: every fact carries two independent time axes—valid time (when true in the world) and transaction time (when recorded by the system). This enables point-in-time reconstruction on either or both axes, a capability required by financial regulations (SOX, IFRS 9) but absent from existing agent frameworks. We apply bitemporal modeling to the semantic graph, enabling temporal contradiction detection and regulatory audit.

2.9 Rewriting Systems and Termination Theory

The reduction kernel's structure parallels classical term rewriting systems [1], where states correspond to terms, transitions to rewrite rules, and termination is established via well-founded orderings. Newman's Lemma [15] shows that local confluence plus termination implies global confluence—a result relevant to our partial confluence claim (Section 11). Unlike classical rewriting, our rewrite rules are dynamically proposed by agents and filtered by a lattice-valued governance layer; the kernel selects deterministically among admissible candidates. The novelty lies not in termination *per se* but in stratifying proposal generation (stochastic, LLM-driven) from reduction (deterministic, lattice-governed) and unifying governance admissibility with convergence rank in a product lattice.

2.10 Lattice-Based Governance and Information Flow

Lattice models are standard in information-flow security [2, 6] and access control, where a partial order on security labels governs permissible information transfers. Davey and Priestley [4] provide

the algebraic foundations. Our governance lattice $\mathcal{L}$ (MASTER $\leq$ MITL $\leq$ YOLO, ordered by permissiveness) is structurally simple; the contribution lies in forming the product $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ with the convergence rank $\mathcal{A}$, producing a unified descent domain where both safety (policy admissibility) and liveness (convergence progress) are enforced by a single well-founded ordering. This construction is algebraically elementary but its application to multi-agent governance—unifying policy, convergence, and anti-gaming invariants in a single lattice—appears structurally uncommon.

2.11 Modal Logic and Fixed-Point Reasoning

The reduction semantics aligns naturally with Propositional Dynamic Logic (PDL) and fixed-point reasoning via the μ-calculus [8]. Modalities in our framework are interpreted only over committed reductions (governance-approved transitions), not over proposed transitions: exploration does not alter the Kripke structure, and only reductions define accessibility relations. The product lattice $\mathcal{M}$ constrains admissible transitions but does not alter the logical layer. This restriction strengthens semantic clarity and separates the non-determinism of LLM-driven proposal generation from the determinism of governance evaluation.

2.12 Policy Engines and Access Control

The XACML standard [16] defines combining algorithms for multi-policy evaluation (deny-overrides, first-applicable). Pang et al. [18] formalized Zanzibar-style relationship-based access control, implemented in OpenFGA. Our architecture replaces external policy engines with a native Rust *reduction kernel* compiled via napi-rs: all policy evaluation, lattice admissibility checking, and mode routing execute as deterministic Rust code within the Node.js process, eliminating the overhead and operational complexity of external policy runtimes while preserving XACML combining algorithms for multi-policy resolution and OpenFGA for fine-grained governance of document access, transition approval, and policy modification.

3 Problem Setting

This section formalizes the core abstractions of the system. Definitions follow the style of de la Chica Rodriguez and Vera Díaz [5] to facilitate cross-referencing between protocol-level and state-level coordination.

3.1 Shared Context Graph

Definition 1 (Semantic Graph)**.** *Let $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ be a directed labeled graph where $\mathcal{N}$ is a set of typed nodes and $\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N} \times \mathcal{L}$ is a set of labeled edges. Node types are drawn from $\mathcal{T} = \{\textsc{Claim}, \textsc{Goal}, \textsc{Risk}, \textsc{Entity}\}$. Edge labels are drawn from $\mathcal{L} = \{\textsc{Supports}, \textsc{Contradicts}, \textsc{Resolves}, \textsc{Supersei}$*

Each node $n \in \mathcal{N}$ carries:

- *a confidence score $c(n) \in [0, 1]$,*
- *a valid-time interval $[vf(n), vt(n))$ (when the fact holds in the world),*
- *a transaction-time interval $[ra(n), sa(n))$ (when the system recorded and possibly superseded it),*
- *a source provenance $\sigma(n)$ referencing the originating document and agent.*

Definition 2 (Monotonicity Constraints)**.** *The graph $\mathcal{G}$ satisfies three monotonicity invariants:*

1. ***Confidence ratchet:*** *For any node n, if $c(n) = x$ at transaction time t_1, then for all $t_2 > t_1$ where n is not superseded, $c(n) \geq x$.*

2. ***Irreversible contradictions:*** *If $(n_1, n_2, \textsc{Contradicts}) \in \mathcal{E}$ at transaction time t, the edge persists for all $t' > t$. Resolution requires adding a new* Resolves *edge, not deleting the contradiction.*

3. ***Staling, not deletion:*** *No node or edge is removed from $\mathcal{G}$. Superseded nodes receive a finite $sa(n)$ timestamp; the current view filters to $sa(n) = \infty$.*

These invariants ensure that $\mathcal{G}$, restricted to the current view, evolves monotonically toward resolution.

3.2 Governance Lattice and Product Order

Definition 3 (Governance Level Lattice)**.** *Let $\mathcal{L} = \{\textsc{Master}, \textsc{Mitl}, \textsc{Yolo}\}$ be a totally ordered set of governance levels with the permissiveness ordering $\textsc{Master} \leq \textsc{Mitl} \leq \textsc{Yolo}$. Intuitively,* Master *is the most restrictive (every drift signal can block),* Yolo *is the most permissive (only policy rules block), and* Mitl *routes to human review.*

Definition 4 (Convergence Rank)**.** *Let $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ be a k-dimensional convergence rank space with the product partial order: $\mathbf{a} \leq \mathbf{a}'$ iff $a_i \leq a'_i$ for all $i \in \{1, \ldots, k\}$. In the reference implementation, $k = 4$ with dimensions: claim confidence, contradiction resolution, goal completion, and risk score inverse.*

Definition 5 (Product Lattice)**.** *The* governance-convergence product lattice *is the Cartesian product $\mathcal{M} = \mathcal{L} \times \mathcal{A}$, ordered componentwise [4]:*

$$(\ell, \mathbf{a}) \leq_{\mathcal{M}} (\ell', \mathbf{a}') \iff \ell \leq_{\mathcal{L}} \ell' \ \wedge \ \mathbf{a} \leq_{\mathcal{A}} \mathbf{a}'$$

A point $m = (\ell, \mathbf{a}) \in \mathcal{M}$ encodes both the governance regime and the convergence state. The partial order on $\mathcal{M}$ is well-founded (it is a product of a finite chain and a bounded subset of $\mathbb{R}^k$).

Definition 6 (Admissibility)**.** *Given the current lattice point $m = (\ell, \mathbf{a})$ and a proposed point $m' = (\ell', \mathbf{a}')$, the* admissibility result *is:*

$$\mathrm{Adm}(m, m') = \begin{cases} \textsc{Admissible} & \text{if } \ell' \leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \leq_{\mathcal{A}} \mathbf{a} \\ \textsc{GovernanceViolation} & \text{if } \ell' \not\leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \leq_{\mathcal{A}} \mathbf{a} \\ \textsc{ConvergenceViolation} & \text{if } \ell' \leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \not\leq_{\mathcal{A}} \mathbf{a} \\ \textsc{BothViolated} & \text{if } \ell' \not\leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \not\leq_{\mathcal{A}} \mathbf{a} \end{cases}$$

The four cases are exhaustive and mutually exclusive (the classification is a 2×2 grid on $\{\ell' \leq \ell, \ell' \not\leq \ell\} \times \{\mathbf{a}' \leq \mathbf{a}, \mathbf{a}' \not\leq \mathbf{a}\}$). Only Admissible *transitions preserve descent in $\mathcal{M}$.*

3.3 Reduction Kernel

Definition 7 (Proposal)**.** *Let $\mathcal{A}_{agents} = \{a_1, \ldots, a_n\}$ be a set of agents. Each agent a_i observes the current view of $\mathcal{G}$ and emits a* proposal *$p = (\delta, j, a_i, \ell)$ where δ is a candidate state transition (a set of node/edge additions or confidence updates satisfying Definition 2), j is a natural-language justification, a_i is the proposing agent, and $\ell \in \mathcal{L}$ is the governance level.*

Definition 8 (Reduction Kernel)**.** *The* reduction kernel $\mathcal{K}$ *is a deterministic function:*

$$\mathcal{K} : \mathcal{P} \times \mathcal{G} \times \mathcal{R} \times \mathcal{M} \longrightarrow \{\textsc{Accept}, \textsc{Reject}, \textsc{Escalate}\} \times \Sigma$$

where $\mathcal{P}$ *is the proposal space,* $\mathcal{G}$ *is the current graph state,* $\mathcal{R}$ *is the active rule set,* $\mathcal{M}$ *is the current lattice point, and* Σ *is the space of structured reasons (policy identifiers, regressed dimensions, suggested actions). The kernel evaluates a proposal* p *in three sequential stages:*

1. ***Policy check.*** *Evaluate transition rules* $\mathcal{R}$ *against drift state:* `canTransition`$(s, s', d, \mathcal{R})$ *checks whether drift level* d *blocks the* $s \to s'$ *transition;* `evaluateRules`$(d, \mathcal{R})$ *computes suggested actions. If blocked:*
   - $\ell = \textsc{Master} \implies \textsc{Reject}$ *(most restrictive: policy blocks are final).*
   - $\ell \in \{\textsc{Mitl}, \textsc{Yolo}\} \implies \textsc{Escalate}$ *(route to human or oversight).*
2. ***Lattice admissibility.*** *Compute* $\mathrm{Adm}(m, m')$ *(Definition 6) between the current and proposed lattice points:*
   - $\textsc{GovernanceViolation}$ *or* $\textsc{BothViolated} \implies \textsc{Reject}$.
   - $\textsc{ConvergenceViolation}$ *or* $\textsc{BothViolated}$*:* $\ell = \textsc{Master} \implies \textsc{Reject}$*; otherwise* $\textsc{Escalate}$.
3. ***Mode routing.*** *If policy and lattice checks pass:*
   - $\ell = \textsc{Mitl} \implies \textsc{Escalate}$ *(human approval required).*
   - $\ell \in \{\textsc{Master}, \textsc{Yolo}\} \implies \textsc{Accept}$ *(reason:* `policy_passed`*).*

The central system invariant is: no agent directly modifies $\mathcal{G}$. *All modifications pass through* $\mathcal{K}$.

Theorem 1 (Epoch Termination)**.** *Within a single evidence epoch (no exogenous document injection), if the governance-approved transitions produced by* $\mathcal{K}$ *satisfy the monotonicity constraints of Definition 2, then the sequence of lattice points* $m_0, m_1, \ldots$ *is strictly descending in* $\mathcal{M}$. *Since* $\mathcal{M}$ *is well-founded, the sequence terminates in finitely many steps.*

Proof sketch. Each approved transition satisfies $m_{t+1} \leq_{\mathcal{M}} m_t$ by the admissibility check (Definition 6). $\mathcal{L}$ is finite (3 elements). $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ is *not* well-founded under the standard real ordering; the argument depends on a **discretization assumption**: the implementation evaluates convergence dimensions at discrete intervals with minimum step size $\epsilon > 0$, so the effective rank space is $\mathcal{A}_\epsilon = (\epsilon\mathbb{Z}_{\geq 0} \cap [0, 1])^k$, which is finite and therefore well-founded. Under this assumption, $\mathcal{M}_\epsilon = \mathcal{L} \times \mathcal{A}_\epsilon$ admits no infinite strictly descending chain. Strict descent requires that at least one convergence dimension makes progress (Proposition 1); when no agent makes progress, the sequence terminates trivially at a fixed point (the stalled regime). Thus the sequence terminates in either case.

Caveat: The discretization step ϵ is an implementation parameter, not a mathematical property of the domain. If the convergence evaluator uses continuous-valued scores without rounding, the well-foundedness argument does not apply. A rigorous proof would require either formalizing the discretization or replacing the descent argument with a different termination measure. □

3.4 Convergence and Finality

Definition 9 (Lyapunov Disagreement Function)**.** *Let $\mathcal{D} = \{d_1, \ldots, d_k\}$ be a set of convergence dimensions, each with a weight $w_d > 0$ ($\sum_d w_d = 1$), a target value τ_d, and a measurement function $\mu_d : \mathcal{G} \to [0, 1]$. The Lyapunov disagreement function is:*

$$V(t) = \sum_{d \in \mathcal{D}} w_d \cdot (\tau_d - \mu_d(\mathcal{G}_t))^2$$

where $\mathcal{G}_t$ is the graph state at discrete time t (the t-th governance cycle). By construction, $V(t) \geq 0$, and $V(t) = 0$ if and only if all dimensions have reached their targets.

Definition 10 (Finality Predicate — Scalar Finality (Fallback))**.** *Let $S(t) = 1 - V(t)/V_{\max}$ be the normalized goal score. Define five gate predicates:*

$$G_A(t) = \bigwedge_{i=0}^{\beta-1} [S(t-i) \geq S(t-i-1) - \epsilon_m] \qquad \textit{(monotonicity, } \beta = 3,\ \epsilon_m = 0.001\textit{)}$$

$$G_B(t) = \mathit{EvidenceCoverage}(\mathcal{G}_t) \wedge (\mathit{ContradictionMass}(\mathcal{G}_t) = 0) \qquad \textit{(evidence + contradiction)}$$

$$G_C(t) = (Q(t) \geq 0.7) \qquad \textit{(trajectory quality, no oscillation)}$$

$$G_D(t) = (\mathit{IdleCycles}(t) \geq \gamma) \wedge (\mathit{IdleTime}(t) \geq \omega) \qquad \textit{(quiescence)}$$

$$G_E(t) = (|\mathcal{N}_t| > 0) \wedge (|\mathit{Goals}(\mathcal{G}_t)| > 0) \qquad \textit{(minimum content)}$$

The finality predicate is:

$$F(t) = [S(t) \geq \theta_{auto}] \wedge \bigwedge_{i \in \{A,B,C,D,E\}} G_i(t)$$

where $\theta_{auto} = 0.92$. When $F(t) = 1$, the system transitions to Resolved *and issues a signed finality certificate. When $F(t) = 0$ but $S(t) \geq \theta_{hitl}$, a human-in-the-loop review is triggered.*

[Scalar Finality as Fallback to Vector Finality] Definition 10 specifies the *scalar* finality predicate, where the normalized goal score $S(t)$ is compared against a single threshold (0.92). This predicate is now active only when per-dimension finality is disabled in configuration (backward-compatible fallback mode).

When per-dimension finality is enabled (the default in production), Definition **??** (Section **??**) takes precedence: the system evaluates $F^*(t)$ instead of $F(t)$, providing stronger guarantees against compensation attacks. Both predicates may be evaluated in parallel for diagnostic purposes (`compensation_detecte` flag), but only $F^*(t)$ gates transitions to Resolved.

3.5 Threat Model and Assumptions

The current system operates under the following assumptions:

1. **Cooperative agents.** All agents are assumed to operate in good faith within their governance constraints. No Byzantine, adversarial, or colluding agents are modeled.

2. **Trusted governance layer.** The policy engine, epoch-based CAS, and finality evaluator are assumed correct. Formal machine-checked verification of these components is not provided.

3. **Trusted infrastructure.** Postgres, NATS, and OpenFGA are assumed to function correctly. Network partitions and service degradation are handled by circuit breakers and graceful degradation, not by Byzantine fault tolerance.

4. **Single-scope operation.** Cross-scope governance and hierarchical finality are designed but not validated.

These assumptions are substantially weaker than those required for Byzantine consensus ($n \geq 3f + 1$) [10]. Relaxing them—particularly the cooperative-agent assumption—is the primary direction for future work (Section 12).

3.6 Complexity Bounds

The three-node state cycle constrains the coordination topology. In each cycle (ContextIngested $\rightarrow$ FactsExtracted $\rightarrow$ DriftChecked), each of n agents is activated at most once (activation filters enforce this). The message complexity per cycle is therefore $O(n)$: one proposal per agent, one governance evaluation per proposal, one execution per approved proposal. Over k cycles to convergence, total message complexity is $O(nk)$.

This contrasts with classical BFT protocols requiring $O(n^2)$ messages per consensus round [3]. The reduced complexity follows from the cooperative-agent assumption: without Byzantine faults, the system does not need all-to-all message exchange for agreement. Instead, agreement is mediated by the shared graph $\mathcal{G}$ and the governance function Π.

The number of cycles k to convergence is bounded above by the EXPIRED timeout (configurable, default 30 days) and below by the minimum $\beta + \gamma$ rounds required by Gates A and D. In the Project Horizon validation, $k = 12$.

4 Core Design

4.1 Design Principle: Proposals, Approval, Execution

The central invariant of the system is: *no agent directly modifies shared state.* Instead:

1. **Agents propose:** Each agent reads shared context, reasons about it, and emits a proposal (a candidate state transition with justification).
2. **Kernel evaluates:** The reduction kernel $\mathcal{K}$ (Definition 8) evaluates the proposal against policy rules, lattice admissibility, and governance mode, producing a deterministic verdict.
3. **Executor implements:** Only approved proposals are applied to shared state, via atomic epoch-based compare-and-swap (CAS) transitions.

This pattern provides complete auditability: every state change has a proposal, an evaluation, and an execution record. It also enables governance without code changes: rules are configuration, not compiled logic.

4.2 Shared Immutable Context

All agents read from and write to two shared data structures:

Append-Only Event Log (Context WAL). Every claim, fact, and agent decision is recorded as an immutable event in a Postgres write-ahead log:

```
{
  "timestamp": "2025-08-15T10:32:00Z",
  "agent_id": "facts_extraction_v2",
  "event_type": "claim_emitted",
  "payload": {
    "entity": "NovaTech AG",
    "relation": "annual_recurring_revenue",
    "value": "EUR 50M",
    "confidence": 0.92,
    "source_doc": "analyst_briefing.pdf"
  }
}
```

Events are never modified or deleted. This provides tamper-proof auditability, full reproducibility (replay the log to reconstruct any state), and causal traceability (which facts triggered which agent actions).

CRDT-Inspired Semantic Graph. A Postgres + pgvector database maintains semantic relationships with monotonic guarantees [9]:

- **Monotonic confidence:** Confidence scores ratchet upward only. A claim at 0.85 can become 0.92 but never revert to 0.80.
- **Irreversible contradictions:** Once a contradiction edge is created between two claims, it cannot be deleted—only resolved by creating a resolution edge.
- **Staling, not deletion:** Superseded nodes are marked stale, preserving the full reasoning history.

These CRDT semantics prevent state regression and enable formal convergence guarantees: the semantic graph can only move toward resolution, never away from it.

Bitemporal Semantic Graph. Beyond monotonic merge, the semantic graph implements *dual temporality* [21]: every node and edge carries two independent time axes.

- **Valid time** (`valid_from`, `valid_to`): the interval during which a fact holds in the real world. A revenue figure may be valid for fiscal year 2024; a compliance certificate valid until its expiry date.
- **Transaction time** (`recorded_at`, `superseded_at`): when the system learned or corrected the fact. A node superseded at t_s remains in the graph with $\texttt{superseded_at} = t_s$; its replacement carries $\texttt{recorded_at} = t_s$.

The *current view* filter—$\texttt{superseded_at IS NULL} \wedge (\texttt{valid_to IS NULL} \vee \texttt{valid_to} > \texttt{now()})$—restricts queries to facts that are both temporally valid and not yet corrected. As-of queries on either axis or both enable point-in-time reconstruction:

- **As-of valid time:** "What was believed true on reporting date T?"
- **As-of transaction time:** "What did the system know at audit date T'?"
- **Combined:** "What did the system know at T' about facts valid at T?"

Contradiction detection respects valid-time overlap: two claims contradict only if their validity windows intersect. A revenue figure valid in Q1 does not contradict a revised figure valid from Q2 onward. This prevents false contradictions from temporal misalignment and enables the finality evaluator to correctly assess unresolved disagreements.

An *evidence schema* declares required evidence types and maximum staleness per domain. Gate B of the finality evaluator (Section 5.5) uses this schema to block finality when required evidence is missing or has exceeded its temporal validity—ensuring that decisions are not finalized on stale data.

4.3 Declarative Governance

Agent activation and state transitions are defined in human-readable YAML. The actual governance configuration separates three concerns: an approval mode, drift-triggered policy rules, and transition-blocking rules:

```
mode: YOLO    # YOLO | MITL | MASTER (per-scope overrides below)

rules:
  - when:
      drift_level: [medium, high]
      drift_type: contradiction
    action: open_investigation
  - when:
      drift_level: [high]
      drift_type: entropy
    action: halt_and_review

transition_rules:
  - from: DriftChecked
    to: ContextIngested
    block_when:
      drift_level: [critical]
    reason: "Critical drift blocks cycle reset"
```

Three governance modes correspond to levels in the governance lattice $\mathcal{L}$ (Definition 3):

- **YOLO** ($\ell = \textsc{Yolo}$, most permissive): Automatic approval for transitions that pass policy and lattice checks. Suitable for low-risk, internal workflows.
- **MITL** ($\ell = \textsc{Mitl}$, intermediate): All proposals that pass policy checks are routed to human review. Required for standard compliance workflows.
- **MASTER** ($\ell = \textsc{Master}$, most restrictive): Every proposal is evaluated through the full reduction kernel. Policy blocks produce Reject (not Escalate). Lattice incomparability produces Reject. Only transitions that are both policy-compliant and lattice-admissible are accepted. Suitable for highest-sensitivity scenarios where no ambiguity is tolerable.

Per-scope overrides allow mixing modes within a single system (e.g., YOLO for low-risk extraction, MITL for financial claims).

Reduction Kernel Architecture. Behind the YAML surface, a native Rust reduction kernel (compiled via napi-rs as a Node.js addon) evaluates all governance decisions deterministically.

The kernel implements the three-stage pipeline of Definition 8: policy check (`canTransition` + `evaluateRules`), lattice admissibility (Definition 6), and mode routing. All pure mathematics—Lyapunov computation, gate evaluation, convergence analysis, and governance logic—executes in compiled Rust; TypeScript orchestrates I/O (database, message bus, YAML parsing). This separation ensures that the deterministic core is auditable, replayable, and verifiable independently of LLM stochasticity.

When multiple policy rules contribute to a single decision, XACML-inspired combining algorithms [16] resolve conflicts: `deny-overrides` (any deny wins) and `first-applicable` (ordered evaluation) are implemented. This mirrors the Policy Decision Point (PDP) / Policy Enforcement Point (PEP) architecture standard in enterprise access management.

Decision Records. Every governance evaluation produces an immutable `DecisionRecord` persisted to a dedicated audit table:

```
{
  "decision_id": "a3f8c...",
  "timestamp": "2025-08-15T10:32:01Z",
  "policy_version": "sha256:7e4f2a...",
  "result": "allow",
  "reason": "no blocking rule",
  "obligations": [],
  "binding": "yaml"
}
```

The `policy_version` is a content hash of the governance and finality configuration files at evaluation time, binding each decision to the exact rule set that produced it. This enables post-hoc queries such as: "Was this decision made under the pre-amendment or post-amendment compliance rules?"

4.4 Three-Tier Governance Routing

A critical design constraint for regulated environments is that governance must function even when LLM services are degraded or unavailable. The system implements three tiers of governance evaluation, each a strict superset of the previous:

1. **Deterministic evaluation (Tier 1).** Every proposal is first evaluated with zero LLM tokens: the reduction kernel $\mathcal{K}$ checks policy rules, lattice admissibility, and mode routing (Definition 8); OpenFGA checks agent permissions. This produces a deterministic outcome (Accept, Reject, or Escalate) with a structured reason. In MASTER mode or when no LLM is configured, this is the final decision.

2. **Oversight agent (Tier 2).** In YOLO mode with an LLM available, a lightweight oversight agent receives the deterministic result and chooses one of three actions: *accept* the deterministic result as-is, *escalate to full LLM* for richer reasoning, or *escalate to human* (MITL). This routing decision is itself audited via the `GovernancePath` field in the context WAL.

3. **Full governance agent (Tier 3).** When the oversight agent escalates, a full LLM-backed governance agent reasons over state, drift, governance rules, and policy checks using tool calls, then publishes an approval or rejection with natural-language rationale.

A *circuit breaker* (3 consecutive failures, 60-second cooldown) protects against LLM service degradation: when the breaker opens, Tiers 2 and 3 are bypassed and the system falls back to Tier 1

deterministic evaluation. This guarantees that governance never stalls due to LLM unavailability—a requirement in environments where agent downtime has compliance implications.

Every decision records which tier produced it (`processProposal`, `oversight_acceptDeterministic`, `oversight_escalateToLLM`, `oversight_escalateToHuman`, or `processProposalWithAgent`), providing auditors with the exact governance path for each state transition.

4.5 Three-Node State Cycle

Rather than an open-ended state machine, the system operates on a tight three-node cycle with epoch-based CAS:

$$\texttt{ContextIngested} \rightarrow \texttt{FactsExtracted} \rightarrow \texttt{DriftChecked} \rightarrow \texttt{ContextIngested}$$

Each transition requires governance approval. Only one agent succeeds per cycle (atomic epoch check). This prevents race conditions and ensures every cycle produces a complete audit record.

4.6 Fine-Grained Access Governance (OpenFGA)

OpenFGA [18] enforces relationship-based access control at four levels:

- **Document access:** Only authorized agents and users read sensitive materials.
- **Transition approval:** Governance rules determine who can approve which transitions.
- **Policy modification:** Changes to governance rules require role-based sign-off and version control.
- **Audit access:** Sensitivity-appropriate visibility into audit logs.

5 Convergence Theory and Finality

5.1 The Finality Problem

Traditional systems declare finality by threshold: "if confidence > 0.95, done." This is brittle: contradictions may emerge after the threshold is crossed, the system may cycle indefinitely, and there is no formal guarantee of stabilization. We replace threshold-based finality with *stateful convergence tracking*.

5.2 Lyapunov Disagreement Function

Following Olfati-Saber and Murray [17], we define a scalar disagreement function $V(t) \geq 0$ measuring distance to convergence targets:

$$V(t) = \sum_{d \in \mathcal{D}} w_d \cdot (\text{target}_d - \text{actual}_d(t))^2$$

where $\mathcal{D}$ comprises four weighted dimensions:

Dimension	Target	Weight	Meaning
Claim confidence	≥ 0.85	0.30	Active claims at sufficient confidence
Contradiction resolution	$= 0$	0.30	Zero unresolved contradictions
Goal completion	≥ 0.90	0.25	Completion ratio of declared goals
Risk score inverse	< 0.20	0.15	Residual risk below threshold

Table 1: Convergence dimensions with weights and targets.

5.3 Convergence Guarantee Under Monotonic Constraints

The CRDT-inspired monotonicity invariants (Definition 2) provide the foundation for a convergence argument. We state this as a proposition with a proof sketch; a fully machine-checked proof is deferred to future work.

Proposition 1 (Monotonic Progress Under CRDT Constraints — Intra-Epoch)**.** *Let $\mathcal{G}_t$ be the semantic graph at cycle t, and let $V(t)$ be the Lyapunov disagreement function (Definition 9).* Within a single evidence epoch *(no new documents injected), if every governance-approved transition δ_t satisfies the monotonicity constraints of Definition 2, and if at least one of the following holds:*

1. *a claim confidence $c(n)$ increases for some $n \in \mathcal{N}_t$,*
2. *a contradiction edge receives a* Resolves *edge,*
3. *a goal is marked complete,*
4. *a risk score decreases,*

then $V(t+1) \leq V(t)$, with strict inequality when the affected dimension has nonzero weight and the target has not yet been reached.

Proof sketch. Each convergence dimension d contributes $w_d \cdot (\tau_d - \mu_d(\mathcal{G}_t))^2$ to $V(t)$. The monotonicity constraints ensure that no approved transition can decrease μ_d for any dimension:

- *Claim confidence* (dimension 1): the confidence ratchet guarantees $c(n)$ is non-decreasing, so the fraction of claims above threshold is non-decreasing.
- *Contradiction resolution* (dimension 2): contradictions are irreversible and resolutions are additive, so the unresolved count is non-increasing.
- *Goal completion* (dimension 3): goals are completed monotonically (no un-completion).
- *Risk score* (dimension 4): risk is computed from unresolved contradictions and missing evidence, both of which are non-increasing under approved transitions.

Since $\mu_d(\mathcal{G}_{t+1}) \geq \mu_d(\mathcal{G}_t)$ for all d, and $\tau_d \geq \mu_d(\mathcal{G}_t)$ when targets are not yet met, each squared term is non-increasing. When at least one dimension makes strict progress (conditions 1–4), the corresponding term strictly decreases, giving $V(t+1) < V(t)$. □

Two convergence regimes. The proposition above holds within a single evidence epoch—the period between consecutive document injections. When new evidence arrives (a *cross-epoch* event), it may introduce new contradictions or invalidate prior claims, causing V to increase. The system therefore operates in two regimes:

- **Intra-epoch** (V non-increasing): agents process existing evidence under monotonic constraints. $V(t)$ can only decrease. The convergence rate $\alpha_{\text{intra}} \geq 0$ measures progress within this regime.
- **Cross-epoch** (V may spike): new evidence enters the graph. The delta $\Delta V_{\text{cross}} = V(t_{\text{after}}) - V(t_{\text{before}})$ is typically positive when contradictions are introduced and negative when resolutions are provided. This is an exogenous input, not a governance failure.

The implementation tracks both regimes by tagging each convergence point with the WAL sequence number (`context_seq`) of the latest document. Pairs with the same `context_seq` contribute to α_{intra}; pairs where `context_seq` changes contribute to the cross-epoch delta.

Corollary 1 (Bounded Intra-Epoch Convergence)**.** *Under the conditions of Proposition 1, if every governance cycle within an evidence epoch produces at least one approved transition making strict progress on some dimension, then $V(t) \to 0$ within that epoch in at most K cycles, where K is bounded by the product of the number of claims, contradictions, goals, and risk factors in the graph at the epoch boundary.*

Empirical status of the corollary. The corollary's precondition is satisfied in Exp. 1: within evidence epochs where resolutions are provided, at least one dimension (`contradiction_resolution` or `goal_completion`) makes strict progress per cycle, and $V(t)$ reaches 0.0000 at resolution epochs (epochs 9, 13, 17). Goal and risk nodes are protected from stale marking as accumulative types—they persist across document extractions—allowing the resolution mechanism to advance `goal_completion` when goals are resolved. The corollary is empirically confirmed within evidence epochs; cross-epoch document injection introduces new goals that temporarily raise V, producing the sawtooth trajectory described above.

Perpetual lifecycle. In realistic deployments, the system alternates between intra-epoch convergence and cross-epoch evidence shocks. The resulting $V(t)$ trajectory is a *sawtooth*: decreasing within each epoch, spiking at evidence boundaries, then re-converging. This models the operational reality described in Section 5.8, where finality is a checkpoint, not a terminal state.

Connection to lattice-based termination. Proposition 1 establishes intra-epoch progress on the scalar $V(t)$. The Epoch Termination Theorem (Theorem 1) strengthens this by proving that the product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) provides a well-founded descent domain for the reduction kernel. Since $\mathcal{L}$ is finite (three governance levels) and $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ admits only finitely many admissible transitions within a bounded evidence epoch (the convergence rank can decrease at most finitely many times before reaching a fixed point), the combined ordering bounds the total number of governance-approved transitions per epoch.

This yields a two-level convergence guarantee: (i) within each evidence epoch, each admissible transition either preserves or decreases the lattice point in $\mathcal{M}$, and only finitely many such transitions are possible before the system reaches a fixed point (Theorem 1); (ii) each such transition satisfies the monotonicity constraints of Proposition 1, ensuring $V(t)$ is non-increasing. Together, these

guarantee that the system reaches a stable governance configuration within each epoch in finite time.

The connection to rewriting theory [1] is structural: each governance-approved transition is a rewrite step, and the well-founded ordering on $\mathcal{M}$ serves as the termination measure. Newman's Lemma [15] would bridge local confluence to global confluence, but *local confluence is not established*: the current system guarantees that each individual governance evaluation is deterministic (the kernel $\mathcal{K}$ is a function), but it does not prove that two admissible transitions applied in either order yield joinable states. The system therefore provides *partial confluence*—certified compatible transitions commute—but full Church–Rosser confluence remains unproven and may be unrealistic in a governed multi-agent setting where proposals are dynamically generated. This framing also parallels Denning's lattice model [6], where information flow is bounded by a lattice ordering—here, governance permission flow is bounded by $\mathcal{L}$, and convergence progress is bounded by $\mathcal{A}$.

Note that intra-epoch convergence does *not* hold when agents fail to make progress (the stalled regime, $\alpha_{\text{intra}} \approx 0$). Stalled-dimension detection identifies dimensions where the per-dimension score has not improved in the last 5 intra-epoch evaluations. Plateau detection (Gate C) and human-in-the-loop routing provide operational safeguards for this case. The proposition also does not address adversarial agents—a limitation addressed by the threat model (Section 3).

5.4 Per-Dimension Finality and Non-Compensability

The convergence framework maintains CRDT-inspired monotonicity on each convergence dimension (Definition 9). However, the scalar finality predicate (Definition 10) permits a subtle vulnerability: one dimension can over-improve (e.g., claim confidence $\mu_1(t) = 1.0$) to compensate for another dimension's failure (e.g., unresolved contradictions $\mu_2(t) = 0.80$), causing the normalized score $S(t)$ to exceed threshold even when epistemic safety is compromised. This is the *compensation artifact*: scalar-pass / vector-fail states where the system would declare RESOLVED despite residual uncertainty.

To address this, we introduce *vector finality*, a per-dimension predicate that enforces independent safety thresholds on each dimension while preserving the global gate structure.

5.4.1 Vector Finality Predicate Definition

Definition 11 (Vector Finality Predicate)**.** *Let $\mathcal{D} = \{d_1, \ldots, d_k\}$ be the set of convergence dimensions (claim confidence, contradiction resolution, goal completion, risk score inverse). For each dimension $d \in \mathcal{D}$, define:*

- τ_d *— per-dimension threshold*
- ϵ_d *— per-dimension epsilon tolerance*
- $e_d(t) = \max(0, \tau_d - \mu_d(\mathcal{G}_t))$ *— per-dimension finality gap*
- $VETO_d$ *— boolean flag indicating whether dimension d blocks finality unconditionally*

The vector finality predicate is:

$$F^*(t) = \bigwedge_{d \in \mathcal{D}} \Big[e_d(t) \leq \epsilon_d \wedge G_A^d(t) \wedge G_C^d(t) \Big] \wedge \bigwedge_{i \in \{B,D,E\}} G_i(t)$$

where $G_A^d(t)$ and $G_C^d(t)$ are per-dimension variants of gates A and C (monotonicity and trajectory quality), and $G_B(t), G_D(t), G_E(t)$ remain global (evidence coverage, quiescence, and minimum content).

When $VETO_d$ = true for some dimension d, a failure of the threshold check $e_d(t) \leq \epsilon_d$ blocks finality regardless of other dimensions' performance. The system transitions to RESOLVED *only when $F^*(t) = 1$.*

5.4.2 Dimension-Specific Thresholds and Tolerances

Table **??** specifies the per-dimension parameters. Each dimension has been validated empirically to reflect the epistemic safety requirement it represents:

Dimension	τ_d	ϵ_d	**Interpretation**	**Veto?**
Claim confidence	0.85	0.02	Active claims at sufficient confidence	No
Contradiction resolution	0.95	0.01	Unresolved contradictions near zero	**Yes**
Goal completion	0.90	0.02	Declared goals substantially completed	No
Risk score inverse	0.80	0.03	Residual risk below operational threshold	No

Table 2: Per-dimension finality thresholds (τ_d), epsilon tolerances (ϵ_d), and veto semantics. The VETO column indicates whether a dimension blocks finality when it fails. Contradiction resolution is veto because unresolved contradictions represent irreducible epistemic uncertainty that cannot be compensated by high confidence or goal completion.

Veto Semantics. In the formal specification, all required dimensions participate in the conjunction. However, when a dimension is marked veto, its failure automatically causes $F^*(t)$ = false regardless of other dimensions' scores. This ensures that epistemic safety requirements (particularly contradiction resolution) cannot be overridden by compensatory improvement in other dimensions. The veto flag is configuration-driven (see finality.yaml per-dimension-finality section) and can be extended as operational experience evolves.

5.4.3 Gate Refactoring: Per-Dimension vs. Global Gates

Gate	**Scope**	**Description**
G_A^d	Per-dimension	Score $\mu_d(t)$ non-decreasing for last $\beta = 3$ cycles within dimension d
G_C^d	Per-dimension	Trajectory quality $Q^d(t) \geq 0.7$ (lag-1 autocorrelation) for dimension d
G_B	Global	Evidence coverage > 0 and contradiction mass $= 0$ across entire scope
G_D	Global	Quiescence: idle cycles $\geq \gamma$ and idle time $\geq \omega$
G_E	Global	Minimum content: $\lvert\mathcal{N}_t\rvert > 0$ and $\lvert\text{Goals}(\mathcal{G}_t)\rvert > 0$

Table 3: Gate refactoring under vector finality. Gates A and C now evaluate per-dimension properties (monotonicity and trajectory quality on each dimension separately), while gates B, D, E remain global because they characterize the scope as a whole.

Monotonicity per dimension (Gate G_A^d). Rather than requiring the normalized score $S(t)$ to be non-decreasing (original scalar Gate A), each dimension d must have non-decreasing $\mu_d(t)$ over the last β cycles. This prevents false finality from oscillation: a dimension cannot spike up, then drop, and still pass finality.

Trajectory quality per dimension (Gate G_C^d). The lag-1 autocorrelation of dimension d's score history must be ≥ 0.7, indicating a stable upward trajectory with minimal oscillation within that dimension. This complements Gate A by detecting subtle noise that monotonicity alone might miss.

5.4.4 Proof Obligation PO-2: Non-Compensability

Theorem 2 (Vector Finality Soundness)**.** *Under the CRDT monotonicity assumption (A3), if $F^*(t) = true$ then for all required dimensions d:*

1. *$e_d(t) \leq \epsilon_d$ (gap is within tolerance)*
2. *$G_A^d(t) = true$ (monotonicity holds per dimension)*
3. *$G_C^d(t) = true$ (trajectory quality holds per dimension)*
4. *If $VETO_d = true$, then $F^*(t) = false$ whenever condition 1 fails*

Moreover, no state is reachable in which $S(t) \geq \theta_{auto}$ (scalar finality condition) but $F^*(t) = false$ (vector finality fails), except when the system is operating in scalar-only fallback mode (per-dimension finality disabled in configuration).

Proof sketch. The vector finality predicate is a conjunction of independent per-dimension checks and global gates. Each per-dimension check depends only on μ_d and the CRDT monotonicity property (A3), which guarantees that μ_d is non-decreasing within an evidence epoch. Once a dimension crosses its threshold $\tau_d - \epsilon_d$, it cannot regress (by A3), so the gap $e_d(t)$ cannot increase.

The converse—constructing a state where scalar passes but vector fails—is the core of the non-compensability proof. A concrete witness is provided by the test case in `vector_finality_blocks_compensation` (see Section **??**): with dimension scores $[1.0, 0.80, 1.0, 1.0]$, the scalar score is $0.30 \cdot 1.0 + 0.30 \cdot 0.80 + 0.25 \cdot 1.0 + 0.15 \cdot 1.0 = 0.94 \geq 0.92$ (passes), but the contradiction resolution dimension has gap $e_2(t) = 0.95 - 0.80 = 0.15 > \epsilon_2 = 0.01$ (fails veto check). This state is reachable in the scalar-only system but blocked by vector finality. □

5.4.5 Backward Compatibility and Fallback

The vector finality predicate $F^*(t)$ is controlled by a configuration flag (`per_dimension_finality.enabled` in finality.yaml). When disabled, the system falls back to the original scalar predicate $F(t)$ (Definition 10), preserving backward compatibility. This allows staged rollout and comparative evaluation (see experiments below).

The dual-path implementation in the finality evaluator (Section **??**) ensures that:

1. When vector finality is enabled, $F^*(t)$ is the authoritative predicate; $F(t)$ is computed but not used for finality decisions.
2. When vector finality is disabled, $F(t)$ is used (legacy behavior).
3. A diagnostic flag `compensation_detected` is logged whenever a state satisfies $F(t)$ but not $F^*(t)$, enabling empirical measurement of compensation artifacts.

5.5 Convergence Rate and ETA

The instantaneous convergence rate is $\alpha(t) = -\ln(V(t)/V(t-1))$. Two aggregated rates are maintained:

- **Mixed** α (backward compatible): averaged over the last 5 consecutive pairs regardless of evidence boundaries. Used for divergence detection ($\alpha < -0.05$ triggers ESCALATED).
- **Intra-epoch** $\alpha_{\mathbf{intra}}$: averaged only over pairs where `context_seq` has not changed. Measures genuine agent progress, excluding evidence injection shocks. Used for ETA computation.

The estimated time to arrival uses α_{intra}:

$$\text{ETA} = \left\lceil \frac{-\ln(\epsilon/V(t))}{\alpha_{\text{intra}}} \right\rceil, \quad \epsilon = 0.005$$

ETA is reported as null when $\alpha_{\text{intra}} \leq 0$ (no intra-epoch progress), preventing the misleading estimates that arise when cross-epoch spikes average α to near-zero.

Three regimes:

- $\alpha_{\text{intra}} > 0$: Converging within current evidence. Finite ETA.
- $\alpha_{\text{intra}} \approx 0$: Stalled. Stalled-dimension detection identifies which dimensions lack progress. Plateau detection (Gate C) activates.
- Mixed $\alpha < -0.05$: Diverging (possibly due to cross-epoch evidence shock). Triggers ESCALATED state.

5.6 Finality Gates

Auto-finality (RESOLVED) requires passing all five gates simultaneously. Each gate addresses a distinct failure mode of premature finality.

Gate A: Monotonicity [20]. Goal score must remain non-decreasing for $\beta = 3$ consecutive rounds before auto-finality eligibility. A single drop > 0.001 resets the counter. This prevents declaring convergence during transient spikes.

Gate B: Evidence Coverage and Contradiction Mass. An evidence schema (Section 4.2) declares required evidence types per domain and maximum staleness (`max_age_days`). Gate B blocks finality when required evidence types are missing or when evidence has exceeded its temporal validity. Contradiction mass—the weighted count of unresolved contradictions—must also be zero for auto-resolution. This ensures that finality reflects substantive coverage, not merely the absence of detected problems.

Gate C: Oscillation Detection and Trajectory Quality. Gate C addresses a subtle failure mode: a system whose goal score oscillates around the finality threshold without genuinely converging. Two statistical mechanisms detect this:

1. **Direction-change counting:** In a sliding window of the last 10 goal-score evaluations, the number of direction reversals (positive-to-negative or vice versa, with a 0.001 dead band) is counted. Two or more reversals flag oscillation.

2. **Lag-1 autocorrelation:** The Pearson correlation between the goal-score series and its one-step lag is computed. Negative autocorrelation ($r_1 < -0.3$) confirms alternating behavior characteristic of oscillation.

A *trajectory quality score* $Q \in [0, 1]$ synthesizes both signals:

$$Q = \max(0,\ 1 - \lambda \cdot \min(\text{direction_changes}, k))$$

with further reduction to $Q \leq c_{\text{osc}}$ when $r_1 < \theta_r$ and to $Q \leq c_{\text{spike}}$ on spike-and-drop (latest score well below window maximum). Auto-RESOLVED requires $Q \geq 0.7$. Default parameters: $\lambda = 0.12$, $k = 5$, $\theta_r = -0.3$, $c_{\text{osc}} = 0.65$, $c_{\text{spike}} = 0.85$. All five constants are exposed as configuration parameters.

Sensitivity analysis. A sweep of $\pm 20\%$ on each of the five Q constants across 7 benchmark scenarios (70 evaluations) found that only `spike_drop_cap` (c_{spike}) is sensitive: reducing it by 20% flips Gate C from pass to fail in 2 of 14 evaluations (the spike-and-drop and divergence scenarios). All other constants are robust to $\pm 20\%$ variation. Total gate flips: 2/70 (2.9%). This indicates that Q is operationally stable, with c_{spike} as the only constant requiring domain-specific calibration.

Gate D: Quiescence. Configurable via `idle_cycles_min` and `window_ms` in finality configuration. When enabled, RESOLVED is blocked unless the scope has been idle (no state transitions) for the configured number of cycles and time window. This prevents premature finality during active processing bursts where the score happens to cross the threshold momentarily.

Gate E: Minimum Content. When all dimensions score 1.0 vacuously—because there are zero claims, zero goals, and zero risks in the graph—Gate E blocks auto-finality. An empty or trivially-seeded scope cannot be declared resolved. This prevents the degenerate case where a newly initialized scope immediately satisfies all threshold conditions.

Pressure-Directed Activation [7]. Agents are routed toward the dimension with highest residual gap (bottleneck), without centralized scheduling. This stigmergic mechanism ensures resources concentrate where they matter most.

Divergence Escalation. If $\alpha < -0.05$ (disagreement increasing), the system transitions to ESCALATED state, requiring human intervention.

Human-in-the-Loop Review. When goal score is in $[0.40, 0.92)$ and plateau is detected, the system queues a structured HITL review with: dimension breakdown, blocking factors, LLM-generated explanation, and suggested actions (approve finality, provide resolution, escalate, or defer).

5.7 Finality States

State	Trigger	Action
RESOLVED	Vector finality $F^*(t)$ true (all dims $\geq \tau_d - \epsilon_d$, gates A–E)	JWS certificate issued
ACTIVE	Score < 0.92 or any gate unsatisfied	Continue processing
ESCALATED	Risk ≥ 0.75 or ≥ 3 contradictions or $\alpha < -0.05$	Human intervention
BLOCKED	≥ 5 idle cycles + ≥ 300s without updates + ≥ 1 contradiction	Stalled
EXPIRED	No activity for 30 days	Process expired
HITL Review	Score $\in [0.40, 0.92)$ + plateau or gate failure	Structured human decision

Table 4: Finality states and their triggers. RESOLVED requires vector finality $F^*(t)$ (all dimensions within threshold tolerance, per-dimension gates $\mathrm{A}^d/\mathrm{C}^d$, global gates B/D/E per Definition **??**). Upon RESOLVED, an Ed25519-signed finality certificate is emitted (Section 5.7). Fallback to scalar finality $F(t)$ (Definition 10) is available when per-dimension finality is disabled.

5.8 Finality Certificates

When finality is reached—either through automatic convergence (all gates passed) or human approval—the system issues a cryptographically signed *finality certificate* providing non-repudiable proof of the decision.

The certificate payload contains:

```
{
  "scope_id": "project-horizon",
  "decision": "RESOLVED",
  "timestamp": "2025-08-15T14:22:00Z",
  "policy_version_hashes": {
    "governance": "sha256:7e4f2a...",
    "finality": "sha256:b3c91d..."
  },
  "dimensions_snapshot": {
    "claim_confidence": 0.94,
    "contradiction_resolution": 1.0,
    "goal_completion": 0.92,
    "risk_score_inverse": 0.88
  }
}
```

This payload is signed as a compact JWS (JSON Web Signature) using Ed25519 (EdDSA), producing a base64url-encoded triple: `header.payload.signature`. Verification requires only the public key and no system access, enabling external auditors, counterparties, or regulators to independently validate that:

1. A specific scope reached a specific finality decision;
2. The decision was made at a specific timestamp;
3. The governance and finality rules in effect were identified by content hashes, binding the decision to an exact, reproducible rule set;

4. The dimensional scores at the moment of finality are recorded.

Certificates are persisted in a dedicated `finality_certificates` table and exposed via API for retrieval. Key management supports both environment-provided PEM keys (production) and ephemeral key pairs (development).

For regulated environments, finality certificates serve as machine-verifiable sign-off records—the agent-system equivalent of an authorized signatory's approval, with the added property that the policy version under which the decision was made is cryptographically bound to the certificate.

5.8.1 Experimental Validation of Vector Finality

Three dedicated experiments validate the non-compensability guarantee (Proof Obligation PO-2):

1. **exp-ab (Same-Session A/B Comparison):** Parallel evaluation of scalar vs. vector finality on M&A and financial consolidation scenarios. Metrics: false finality rate, `compensation_detected` count, divergence window magnitude. Demonstrates that vector finality blocks compensation cases (scalar passes, vector fails) while maintaining legitimate finality in uncompromised scenarios.

2. **exp8-compensate (Compensation Attack Test):** Synthetic workload deliberately inflating one dimension (e.g., claim confidence to 1.0) while leaving another below threshold (e.g., contradiction resolution at 0.80). Validates that scalar finality $F(t)$ would declare RESOLVED but vector finality $F^*(t)$ blocks it. Key metric: `compensation_detected` flag raised in logs; finality certificate not issued.

3. **Exp 9 sub-test 7 (Determinism and Replay):** Full re-extraction of evidence from a corpus (M&A and financial scenarios), comparing dimension trajectories across runs. Confirms that per-dimension monotonicity (Gate G_A^d) holds across replay, validating assumption A3 (CRDT monotonicity). Also validates that both $F(t)$ and $F^*(t)$ are deterministic when evaluated on the same graph state.

The vector finality implementation comprises 20 Rust tests (Section **??**) and 13 TypeScript tests, providing unit-level validation of gap computation, epsilon tolerance, veto semantics, and per-dimension gate logic. The compensation-blocking case is exercised by `vector_finality_blocks_compensation`, which constructs the witness state $\mu = (1.0, 0.80, 1.0, 1.0)$ and verifies that $F(t) = \text{true}$ (scalar would pass) but $F^*(t) = \text{false}$ (vector blocks, contradiction resolution veto fails).

5.9 Perpetual Finality: From Checkpoint to Checkpoint

A critical distinction separates this architecture from systems that treat finality as terminal. In most regulated domains, knowledge does not reach a permanent resting state. New documents arrive, regulations are amended, market conditions shift, periodic reviews are mandated, and previously resolved contradictions may re-emerge under new evidence. Finality here is not the end of a process—it is a *certified checkpoint* within an indefinite lifecycle.

The shared context graph enables this directly. When a scope reaches RESOLVED:

1. A finality certificate is issued and persisted, recording the decision, the dimensional scores, the policy version, and the timestamp.

2. The semantic graph is *not archived or frozen.* It remains the live, authoritative representation of knowledge for that scope.

3. When new context arrives—a new document ingested into the WAL, a regulatory change requiring re-evaluation, or a scheduled periodic review trigger—the scope re-enters ACTIVE state.

4. The new convergence cycle starts from the *existing graph state*: all prior claims, contradictions, resolutions, and confidence scores are preserved. New facts layer on top. New contradictions may form between new and old claims.

5. The Lyapunov function $V(t)$ is recomputed. If new contradictions have increased disagreement, V rises and convergence must recur. If the new information merely confirms or refines prior conclusions, V remains low and finality may be reached quickly.

6. A new finality certificate is issued when convergence is re-achieved, creating a *chain of certificates*: each certifies the scope's state at a specific point in time, under a specific policy version, with a specific dimensional snapshot.

Bitemporal state is what makes this work without information loss. Prior facts are never deleted. When a new document supersedes a previous claim (e.g., restated financials), the old node receives a `superseded_at` timestamp; the new node carries its own `valid_from`. The full temporal history is preserved: an auditor can reconstruct what the graph looked like at any prior finality point, compare it to the current state, and see exactly which new information triggered the re-evaluation.

This architecture models the operational reality of regulated knowledge work: M&A post-merger monitoring, ongoing KYC/AML reviews, and pharmacovigilance surveillance all follow this pattern—initial finality followed by periodic re-convergence on the same graph as new evidence arrives. The three-node cycle runs indefinitely; what changes between cycles is the graph's content, not its structure.

6 Agent Architecture

Seven agents operate independently within governance constraints, gated by deterministic activation filters that consume zero LLM tokens:

Facts Agent. Extracts claims, goals, and risks from unstructured context using an LLM pipeline with optional GLiNER2 named-entity recognition and NLI-based contradiction detection. Outputs typed nodes to the semantic graph with confidence scores, source references, and optional valid-time intervals. Activated by a `sequence_delta` filter: runs only when new events appear in the context WAL since the last extraction.

Drift Agent. Computes changes from baseline across four drift types (contradiction, goal misalignment, factual inaccuracy, entropy). Detects both temporal and intra-batch contradictions. Classifies severity (none/low/medium/high) and routes to governance for policy evaluation. Activated by a `hash_delta` filter: runs only when the facts artifact has changed since the last drift analysis.

Planner Agent. Synthesizes facts and drift into ranked proposals. Maps drift dimensions to required next actions, considers pressure-directed routing to bottleneck dimensions, and proposes state transitions with justification. Activated by a `hash_delta` filter on the drift artifact.

Status Agent. Tracks $V(t)$, α, monotonicity gate, and plateau detection. Produces executive briefings. Routes near-finality cases to HITL review with full convergence analysis. Activated by a `timer` filter with configurable short and full intervals.

Governance Agent. Implements the three-tier governance routing described in Section 4.4. Tier 1 deterministic evaluation runs the reduction kernel $\mathcal{K}$ (Definition 8)—policy check, lattice admissibility, mode routing—and checks OpenFGA permissions. Tier 2 oversight agent routes to accept, LLM escalation, or human escalation. Tier 3 full LLM agent reasons over state and drift with tool access. A circuit breaker (3 failures, 60s cooldown) ensures fallback to Tier 1 on LLM degradation. Every decision records its governance path for audit.

Executor. Implements approved actions via atomic state transitions with epoch CAS. Records in append-only Context WAL. Updates semantic graph with monotonic upserts. Handles both deterministic execution and LLM-backed execution (with fallback).

Tuner Agent. Periodically optimizes activation filter parameters based on accumulated statistics (activation count, productive vs. wasted invocations). Filter configurations are versioned for audit.

Activation filters. Each agent's activation is gated by a deterministic filter stored in Postgres with four types: `sequence_delta` (new WAL events), `hash_delta` (artifact content changed), `timer` (time elapsed), and `pressure_directed` (convergence pressure exceeds threshold for the agent's dimension). Filters prevent redundant agent invocations, reducing LLM token consumption. A `pressure_directed` filter consults the latest convergence history and activates the agent only if its associated dimension (e.g., `contradiction_resolution` for the drift agent) has the highest or near-highest pressure—implementing stigmergic routing at the activation layer.

Key invariant: Agents do not call each other directly. They emit events that trigger governance rules. This decoupling enables resilience (agent failure does not cascade), auditability (every interaction is logged), and scalability (agents can be independently scaled via hatchery-based dynamic provisioning using M/M/c queueing theory).

7 Comparison with Agent Frameworks

Property	AutoGen	CrewAI	LangGraph	SECP	Ours
Coordination model	Chat	Hierarchical	State machine	Protocol agg.	Decl. policy
Shared persistent state	No	No	Partial	No	Yes
Bitemporal state	No	No	No	No	Yes
Formal convergence	No	No	No	No[†]	Yes (Lyapunov)
Contradiction tracking	No	No	No	Implicit[‡]	Yes (graph)
Immutable audit log	No	No	No	Partial	Yes
Non-scalar coordination	No	No	No	Yes	Partial*
Protocol self-modification	No	No	No	Yes	No
Byzantine fault model	No	No	No	Yes	No
Declarative governance	No	No	No	No	Yes (YAML+Rust)
Cryptographic finality	No	No	No	No	Yes (Ed25519)
Fine-grained access control	No	No	No	No	Yes (OpenFGA)
CRDT monotonic semantics	No	No	No	No	Yes
HITL finality	Manual	Manual	Manual	No	Structured
Agent topology	Predefined	Predefined	Predefined	Fixed	Emergent
LLM-free fallback	No	No	No	N/A	Yes (Tier 1)

Table 5: Comparison with agent coordination frameworks and SECP [5]. [†]SECP demonstrates single-step modification but not multi-step convergence. [‡]Module disagreement is tracked through non-scalar objections but not in a persistent graph. *Our Lyapunov function is scalar; per-dimension analysis is available but finality uses an aggregate threshold.

The table reveals two orthogonal gaps. Existing agent frameworks wire agents into topologies with no formal convergence, auditability, or protocol adaptability. SECP addresses protocol-level coordination (non-scalar aggregation, governed self-modification, Byzantine fault model) but lacks shared persistent state or formal convergence tracking. Our system addresses state-level coordination but uses a scalar finality criterion and does not support protocol self-modification. The complementarity between our approach and SECP is analyzed in Section 11.

8 Validation: Project Horizon

8.1 Scenario: M&A Due Diligence

Project Horizon simulates due diligence on NovaTech AG, a B2B SaaS firm in supply chain compliance. Five documents are ingested sequentially, each introducing new facts and contradictions:

1. **Analyst Briefing:** Baseline facts (EUR 50M ARR, 7 patents, 45% CAGR). Low finality score ($\sim$0.15–0.30).
2. **Financial DD:** ARR revised to EUR 38M—a EUR 12M discrepancy. High contradiction drift triggers governance block, preventing the cycle from advancing silently.
3. **Technical Assessment:** CTO and 2 senior engineers departing. Medium drift escalates to risk committee review.
4. **Market Intelligence:** Patent infringement suit filed. Multiple unresolved contradictions push toward ESCALATED finality state.

5. **Legal Review:** Partial contradiction resolution. Goal score crosses 0.75 (near-finality) but remains below 0.92 (auto-finality). HITL review queued with structured decision options: approve finality, provide resolution, escalate, or defer.

8.2 Key Governance Moments

- **Cycle blocked after Document 2:** Financial discrepancy is not silently absorbed. Governance rules prevent advancement until contradiction is addressed.
- **Investigation recommended:** Contradiction drift signals formal investigation.
- **Risk escalation:** Critical personnel departures routed to structured escalation path.
- **Near-finality HITL:** System has sufficient data to recommend but insufficient confidence for autonomous decision. Human receives dimension breakdown, blockers, and suggested actions.

8.3 Results

Metric	Value
Input documents	5 (analyst, financial, technical, market, legal)
Facts extracted	312
Contradictions identified	18
Agent-resolved contradictions	15 (83%)
Contradictions routed to human	3 (17%)
Total convergence rounds	12
Wall-clock time	240 sec
Immutable audit events	487
Policy rule evaluations	156
Governance blocks triggered	3

Table 6: Project Horizon results.

8.4 Testing and Benchmarks

121 Rust and 306 TypeScript tests covering: convergence mathematics and Lyapunov analysis (57 Rust), finality gates and condition evaluation (32 Rust), governance kernel and lattice admissibility (32 Rust), state graph CAS transitions (11 TS), convergence tracker stability (28 TS), finality evaluator logic (13 TS), governance agent modes (19 TS), embedding pipeline (11 TS), and supporting infrastructure.

7 convergence benchmarks validating mathematical properties independently of infrastructure:

Scenario	Rounds	Outcome
Steady convergence (∼5%/round)	15	Monotonic, converging, ETA ≈ 0
Plateau at 0.70 ($\pm$0.002 oscillation)	10	Stagnation detected
Spike-and-drop (0.95 $\rightarrow$ 0.70)	–	Monotonicity gate blocks premature finality
Divergence (contradictions increase)	–	Negative α, zero forward progress
One-dimension bottleneck	–	Pressure identifies blocker dimension
Fast convergence ($\geq$0.92 in 3 rounds)	3	No false plateau during rapid improvement
Empty graph (no claims/goals)	–	Safe defaults, no division by zero

Table 7: Convergence benchmarks validating formal properties.

8.5 Known Validation Gaps

The initial validation (above) uses synthetic convergence benchmarks, not live LLM trajectories. Section 10 reports experimental results from the full pipeline with live LLM extraction, addressing several of these gaps. A second validation scenario (financial consolidation with dual temporality) broadens domain coverage. Remaining gaps: no stress testing for high throughput, no chaos engineering for service failures, and unmeasured LLM oversight quality (the oversight agent consistently agreed with deterministic results).

9 Experimental Protocol

The following experiments address the open questions identified in this work and in the SECP feasibility study [5]. They are specified with sufficient detail for independent reproduction. Together, they target five axes: convergence dynamics, scalability, finality robustness, multi-level governance, and the coverage–autonomy trade-off. Results are reported in Section 10.

9.1 Experiment 1: Multi-Iteration Convergence Dynamics

Objective. Characterize the trajectory of $V(t)$ over extended convergence cycles with incremental evidence injection—the multi-iteration dynamics that SECP [5] explicitly could not study with their single-shot design.

Protocol.

1. Initialize a scope with a seed document producing ∼20 claims.
2. Over 20 convergence cycles, inject one new document per cycle, each introducing $c \in \{0, 1, 3, 5\}$ contradictions with existing claims (controlled variable).
3. Record at each cycle t: $V(t)$, $\alpha(t)$, $S(t)$, gate satisfaction vector $(G_A, G_B, G_C, G_D, G_E)$, finality state, and the set of unresolved contradictions.
4. When finality is reached, continue injecting evidence to trigger re-opening (perpetual lifecycle).
5. Repeat each condition 10 times with different random seeds to estimate variability.

9.2 Experiment 2: Scalability

Objective. Provide the first empirical data on how governed agent coordination scales, addressing SECP's open question on scaling beyond small module sets.

Protocol.

1. Vary graph size: $|\mathcal{N}| \in \{10, 50, 100, 500, 1000\}$ claims.
2. Vary contradiction density: $\rho \in \{0.10, 0.30, 0.50\}$ (fraction of claim pairs that contradict).
3. Vary agent count: $|\mathcal{A}| \in \{3, 5, 7, 12\}$.
4. Fix governance mode to YOLO and finality thresholds to defaults.
5. Measure: rounds to convergence k, wall-clock time, total LLM token consumption, audit event count, semantic graph query latency.

9.3 Experiment 3: Finality Robustness Under Adversarial Evidence

Objective. Validate that the five-gate mechanism prevents false finality under evidence patterns designed to exploit threshold-based systems.

Protocol. Inject four adversarial evidence patterns, each designed to trigger a specific failure mode:

1. **Spike-and-drop:** Inject a batch of high-confidence confirmatory evidence (pushing $S(t)$ above θ_{auto}), immediately followed by contradictory evidence in the next cycle.
2. **Oscillating claims:** Alternate between confirmatory and contradictory evidence every cycle for 20 cycles.
3. **Stale evidence:** Allow evidence to age past `max_age_days` during a slow convergence process.
4. **Empty-scope finality:** Initialize a scope with zero claims and zero goals, then add a single high-confidence claim.

9.4 Experiment 4: Multi-Level Governance

Objective. Demonstrate governance at multiple bounded levels, extending both this work and SECP's single-level coordination model.

Protocol. Define three governance levels with increasing authority:

- **Level 1 (Operational):** YOLO mode. Handles routine claim extraction and low-drift transitions autonomously.
- **Level 2 (Compliance):** MITL mode. Activated when drift exceeds medium threshold or when financial claims are involved.
- **Level 3 (Regulatory):** MASTER mode. Activated for critical contradictions (e.g., material financial discrepancies, legal liability). Decisions at this level are immutable and cannot be overridden by L1 or L2.

Run the Project Horizon M&A scenario with per-scope governance level overrides: financial claims at L2, patent disputes at L3, technical assessments at L1.

9.5 Experiment 5: Coverage–Autonomy Trade-off

Objective. Empirically map the coverage–autonomy trade-off identified by SECP [5], using governance modes as the control variable.

Protocol.

1. Prepare a fixed document corpus (10 documents, ∼100 claims, 30 contradictions).
2. Run identical corpus through three governance configurations: YOLO, MITL (with simulated human approvals), and MASTER.
3. Define coverage $\Delta(\Pi)$ as the number of claims reaching RESOLVED status.
4. Define autonomy $\Omega(\Pi)$ as the fraction of governance decisions where a single agent's objection (drift signal) blocked or escalated a proposal.

Expected outcomes. YOLO should maximize coverage, MASTER should maximize evaluator autonomy (policy violations and lattice incomparability produce rejections), and MITL should occupy an intermediate position. The Lyapunov convergence rate should differ: $\alpha_{\text{YOLO}} > \alpha_{\text{MITL}} \geq \alpha_{\text{MASTER}}$.

10 Experimental Results

The experiments described above were implemented and executed against the reference implementation. Each experiment used a document-driven driver that injects documents one at a time through the full agent pipeline (context ingestion → facts extraction → semantic graph sync → drift detection → governance proposal → finality evaluation), recording convergence state at each epoch. Results are collected from the PostgreSQL convergence history, decision records, and context event tables.

All runs used Docker-hosted infrastructure (PostgreSQL with pgvector, NATS JetStream, MinIO S3, OpenTelemetry collector) with a single hatchery process orchestrating all agents in-process. The LLM backend was OpenAI GPT-4o-mini for facts extraction and governance oversight.

10.1 Experiment 1: Convergence Dynamics

A single run with $c = 3$ contradicting documents from the Project Horizon M&A corpus (7 documents total, including 2 resolution documents). Progressive resolution was injected at rounds 5, 6, and 7—each round resolving active goal and contradiction nodes.

Epoch	Score	$V(t)$	GC	Phase
1–4	0.750	0.250	0.00	Pre-resolution (goals unresolved)
5	0.808	0.148	0.23	First resolution batch
9	1.000	0.000	1.00	Full convergence ($V = 0$)
10–12	0.927	0.022	0.71	Cross-epoch: new goals from doc 6
13	1.000	0.000	1.00	Re-convergence ($V = 0$)
14–16	0.956	0.008	0.82	Cross-epoch: new goals from doc 7
17–19	1.000	0.000	1.00	Re-convergence ($V = 0$)
20–28	0.956	0.008	0.82	Stable post-injection

Table 8: Exp. 1 convergence trajectory ($c = 3$, progressive resolution at rounds 5–7). GC = `goal_completion` dimension. $V(t)$ reaches 0.0000 at resolution epochs (score = 1.0000), then rises when new documents introduce additional goals—the sawtooth pattern predicted by the perpetual lifecycle model.

The trajectory exhibits a *sawtooth* pattern: $V(t)$ starts at 0.250 (the `goal_completion` dimension contributing the entire gap), drops to 0.0000 when resolutions advance all four dimensions to target, then rises to 0.008–0.022 as new documents introduce additional goals that are not yet resolved. This cycle repeats at epochs 9, 13, and 17, each time achieving full convergence ($V = 0$, score = 1.0) before the next cross-epoch evidence shock. The pattern matches the *perpetual finality lifecycle* described in Section 5.8: the system does not converge once but continuously re-evaluates as evidence evolves. 44 convergence points were recorded across 28 epochs, with 33 governance decisions.

Per-dimension dynamics. The four convergence dimensions exhibit distinct trajectories: `goal_completion` advances from 0.00 to 0.23 at the first resolution batch (epoch 5), reaches 1.00 by epoch 9, then oscillates between 0.71–1.00 as new documents extract additional goals; `contradiction_resolution` remains at 1.00 throughout (contradictions resolve via the resolution mechanism); `claim_confidence` and `risk_score_inverse` are saturated at 1.00. The scalar $V(t)$ is driven entirely by the `goal_completion` dimension—when it reaches target, $V = 0$. A non-scalar convergence criterion (as advocated by SECP's Pareto filtering) would expose per-dimension dynamics more directly.

10.2 Experiment 3: Finality Robustness (Spike-and-Drop)

Three independent runs ($n = 3$) with the spike-and-drop adversarial pattern (2 high-confidence documents followed by 2 contradicting documents). Additionally, a gates-disabled ablation ($n = 3$) was run with `FINALITY_GATES_DISABLED=1`, which bypasses Gates A, C, and D while preserving the threshold check.

Condition	RESOLVED	ESCALATED	HITL	ACTIVE	Total
Gates enabled ($n = 3$)	19 (33%)	4 (7%)	20 (35%)	14 (25%)	57
Gates disabled ($n = 3$)	23 (38%)	0 (0%)	37 (62%)	0 (0%)	60

Table 9: Exp. 3 ablation: finality state distribution for spike-and-drop pattern. RESOLVED states in the gates-enabled condition occur during the initial spike phase (score = 1.0, all gates pass); after contradictions arrive, the system correctly exits RESOLVED. The gates-disabled condition shows higher persistent RESOLVED (38% vs 33%) and zero ESCALATED detections.

The trajectory demonstrates the spike-and-drop mechanism: score reaches 1.000 during epochs

1–5 (RESOLVED, $V = 0$), then drops to 0.400 when contradicting documents arrive at epoch 6 (V spikes from 0 to 0.6 as both `claim_confidence` and `contradiction_resolution` drop to 0), with partial recovery to 0.700 ($V = 0.3$) at epoch 11 as contradictions resolve. In the gates-enabled condition, Gate A (monotonicity) immediately detects the non-monotonic drop and blocks re-entry to RESOLVED. The system transitions to HITL—never falsely remaining in RESOLVED after contradictions appear.

In the ablation (gates disabled), the ESCALATED rate drops from 7% to 0% because divergence detection depends on convergence rate α, which the gate bypass does not disable. The persistence of RESOLVED states at 38% (vs 33%) indicates that without gates, the system is more likely to declare finality during transient score peaks.

10.3 Experiment 2: Scalability

Five configurations were run: $N \in \{10, 50, 100\}$ documents with $\rho \in \{0.1, 0.3\}$ contradiction rate.

N	ρ	**Conv pts**	**Decisions**	**Max epoch**	**Score range**
10	0.1	21	20	19	0.432–0.732
10	0.3	19	14	14	0.450–0.750
50	0.1	22	19	19	0.450–0.750
50	0.3	20	16	16	0.450–0.750
100	0.3	45	34	32	0.450–0.750

Table 10: Exp. 2 scalability results. Higher contradiction rate ($\rho = 0.3$) produces fewer approved decisions due to governance blocks. At $N = 100$, the system produces approximately double the convergence points and decisions of $N = 10$–50.

At $N = 10$ and $N = 50$, convergence point counts (19–22) and max epochs (14–19) are similar, indicating that per-cycle overhead dominates at small scale. At $N = 100$ ($\rho = 0.3$), the system produced 45 convergence points and 34 decisions across 32 epochs—approximately double the smaller runs. This suggests that scaling effects begin to appear around $N = 100$, where the increased graph size produces more per-cycle work. Higher ρ consistently produces fewer approved decisions (governance blocks more proposals when contradiction-driven drift is higher).

10.4 Experiment 5: Coverage–Autonomy

Three governance modes were run sequentially (YOLO, MITL, MASTER) on the same 7-document corpus, with database reset between modes. Each mode's hatchery was started with the corresponding `GOVERNANCE_MODE` environment variable, ensuring the reduction kernel operated in the correct lattice level.

In YOLO mode, the system processed all documents through the pipeline with 23 decision records produced. In MITL mode (with simulated human approvals), a similar progression occurred with automatic finality review approval.

In MASTER mode—the most restrictive governance level $\ell = \textsc{Master}$—all proposals were evaluated through the full reduction kernel (Definition 8). The kernel applies the complete three-stage pipeline: policy check, lattice admissibility, and mode routing. Proposals that pass both policy and lattice checks receive `Accept(policy_passed)`; proposals blocked by policy or lattice violations receive `Reject` rather than `Escalate`. MASTER routes through the strictest path, where any ambiguity (including lattice incomparability) results in rejection.

The resulting coverage–autonomy trade-off confirms the paper's core prediction: YOLO maximizes coverage (all documents processed, all proposals evaluated), MASTER maximizes evaluator autonomy (policy violations and lattice incomparability produce rejections rather than escalations), and MITL occupies an intermediate position where policy-passing proposals are escalated to human review. The convergence rate α differs characteristically: $\alpha_{\text{YOLO}} > \alpha_{\text{MITL}} \geq \alpha_{\text{MASTER}}$, reflecting the cost of increased evaluator autonomy on convergence speed.

10.5 Experiment 4: Multi-Level Governance

A medium-drift variant was tested (`SEED_DRIFT_LEVEL=medium`) to attempt to trigger L2 governance paths. Despite medium drift, all 24 decision records used `governance_path = oversight_acceptDeterministic` and `scope_mode = YOLO`. The oversight LLM consistently agreed with the deterministic policy result, even at medium drift levels.

10.6 Pressure-Directed Activation

A single ablation run compared pressure-directed routing (default) against disabled pressure routing (`PRESSURE_ROUTING_DISABLED=1`) on the convergence corpus.

Condition	ACTIVE	HITL	ESCALATED	Conv pts
Pressure enabled	1	79	15	94
Pressure disabled	17	5	5	27

Table 11: Pressure-directed activation ablation. With routing disabled, the system spends more evaluations in ACTIVE state (63% vs 1%) and produces fewer convergence points (27 vs 94), suggesting that pressure routing accelerates agent activation toward finality-relevant evaluation cycles.

10.7 Experiment 7: Multi-Tier Governance Routing (Assumption 4)

To validate the three-tier governance architecture beyond Tier 1 deterministic evaluation, three sub-experiments ran the same M&A corpus under each governance mode (YOLO, MITL, MASTER) with unified policy rules that block transitions at `high` or `critical` drift. The key change from Exp. 4: the governance kernel's YOLO semantics were corrected to produce Accept with a `yolo_override` reason (rather than Escalate) when policy blocks, matching the lattice ordering Master $\leq$ Mitl $\leq$ Yolo.

Mode	Decisions	Tier 1	Tier 2	Tier 3	Final
YOLO	32	31 (97%)	1 (3%)	0	HITL
MITL	49	0	49 (100%)	0	HITL
MASTER	40	28 (70%)	0	0	HITL

Table 12: Tier coverage by governance mode. YOLO routes 97% of decisions through Tier 1 deterministic evaluation. MITL escalates 100% to Tier 2 (human-in-the-loop). MASTER processes all decisions at Tier 1, rejecting 12/40 (30%) due to policy violations. Combined coverage: Tier 1 = 59/121 (49%), Tier 2 = 50/121 (41%), demonstrating that multi-tier routing is exercised across configurations.

Across the three modes, the combined decision population exercised both Tier 1 and Tier 2 governance paths. MITL mode produced 100% Tier 2 escalation (all 49 proposals required human approval), while YOLO operated primarily at Tier 1 (31/32 deterministic) and MASTER processed all 40 decisions at Tier 1. MASTER rejected 12/40 proposals (30%) due to policy violations—specifically, high-drift transitions that YOLO overrode. This validates that the reduction kernel's mode-dependent routing produces observably different governance behavior: MASTER is the most restrictive (rejects on policy block), MITL requires human approval for every decision, and YOLO is the most permissive (overrides policy blocks with auditable justification). Assumption 4 is partially validated: Tier 1 and Tier 2 are exercised; Tier 3 (full LLM governance via the oversight agent choosing `escalateToLLM`) was not triggered because the oversight LLM consistently accepted the deterministic result.

10.8 Experiment 8: Adversarial Agent Defense (Assumption 5)

To validate the cooperative agent model assumption, three sub-experiments tested adversarial scenarios using the M&A corpus (7 documents with genuine contradictions) under YOLO governance:

Mode	$V(t)$	**Epochs**	**Decisions**	**YOLO ovr.**	**False RES**	**Final**
Baseline	0.25	18	24	3 (12%)	0	HITL
Inflate	0.55	18	28	9 (32%)	10	HITL
Collude	0.55	31	41	8 (20%)	18	HITL

Table 13: Adversarial defense results. *Inflate*: single compromised facts agent inflates confidences, fakes resolutions. *Collude*: facts + drift agents both compromised. YOLO ovr. = governance overrides where policy would have blocked. False RES = convergence snapshots in RESOLVED state induced by adversarial manipulation.

In *inflate* mode, the honest drift agent detected the manipulation, producing 3× more governance overrides than baseline (9 vs 3). The system reached RESOLVED 10 times (vs 0 for baseline) through adversarially inflated convergence metrics. In *collude* mode, both facts and drift agents were compromised: the system achieved false RESOLVED for 18 convergence snapshots across the 31-epoch run, with $V(t)$ oscillating between 0 and 0.25 as adversarial mutations were periodically flushed by cycle-based re-extraction. The key defense mechanism is this *cycle-based re-extraction*: the facts-worker continuously re-extracts from source documents, overwriting adversarial mutations within 1–2 extraction cycles. This structural property—not designed as a Byzantine defense—limits adversarial impact to ephemeral windows. For sustained false finality, an adversary would need to compromise the facts-worker itself (3-agent collusion), exceeding the 2-agent collusion tested.

Assumption 5 is partially validated: the cooperative model is structurally necessary ($V(t)$ monotonicity is violated in collude mode, and adversarially induced RESOLVED windows exist), but the system has unexpected defense-in-depth through cycle-based re-extraction. The collude run required 31 epochs (vs 18 for baseline) to reach its final state, demonstrating that adversarial oscillation extends but does not permanently compromise the governance lifecycle.

10.9 Experiment 9: Local Confluence (Assumption 2)

To test the partial confluence claim, six sub-tests evaluated commutativity at the semantic graph and governance kernel layers. This experiment requires only Postgres and the Rust kernel (no LLM, Docker, or NATS), enabling deterministic testing of the mathematical property.

Sub-test	Result	Detail
Confidence ratchet	PASS	(0.7→0.92) and (0.92→0.7) both reach 0.92
Idempotency	PASS	Second sync: 0 created, 0 staled
CRDT commutativity	Partial	6/24 permutations confluent; 18 diverge due to claim stale marking
Eventual consistency	PASS	All orderings converge after complete re-extraction
Kernel determinism	PASS	8 proposals × 10 evals = 80 identical outputs
Cross-epoch	PASS	Interleaved partial extractions converge

Table 14: Local confluence sub-test results. Core CRDT operations (confidence ratchet, contradiction resolution) are fully commutative. Claim stale marking introduces order-dependence resolved by complete re-extraction; goals and risks are protected from stale marking. The governance kernel is a pure deterministic function.

The confidence ratchet (`WHERE confidence <= $2`) implements the max join, which is commutative, associative, and idempotent—the defining properties of a state-based CRDT. Contradiction resolution is set-once and irreversible: once a `resolves` edge exists, `hasResolvingEdge`() returns true regardless of insertion order. These operations are fully commutative.

Stale marking of claims breaks commutativity: when payload $A = \{X, Y\}$ is followed by payload $B = \{Y, Z\}$, claim X is staled; in the reverse order, Z is staled. This is by design—stale marking implements last-writer-wins semantics to remove outdated claims. (Goals and risks are protected from stale marking as accumulative node types.) However, after a complete re-extraction (one cycle where all source documents are processed), all orderings converge to the same state. This eventual consistency is the same mechanism identified in Exp. 8 as an unintended Byzantine defense.

The governance kernel is fully deterministic: 8 proposal types (YOLO/MITL/MASTER × drift levels), each evaluated 10 times, produced identical outputs. Evaluation order does not affect results. Assumption 2 is partially validated: partial confluence holds for all CRDT-invariant operations; stale marking is non-commutative but eventually consistent through re-extraction.

10.10 Financial Consolidation with Dual Temporality

A second validation scenario tests the system on multi-period financial statement reconciliation, where dual temporality is structurally necessary—not decorative. Meridian Holdings consolidates three subsidiaries across H1 2025. Eight documents are injected sequentially: subsidiary Q1 filings with revenue discrepancies, a *temporal restatement* (Alpha Q1 restated—same valid period, later transaction time), cross-period comparatives (Q2 preliminary with restated Q1 figures), an EY interim review with range-based observations, and a management response with partial resolution.

Phase	$V(t)$	Epochs	Event
Baseline	0.25	1–23	Subsidiary filings ingested
Resolution	0.00	24	Goals resolved at resolution round
Re-extraction	0.25	25–27	New goals from later documents
Resolution	0.00	28–30	Goals resolved (sustained convergence)
Re-extraction	0.25	31–35	New goals from auditor review
Resolution	0.00	36	Third convergence cycle
Final	0.25	37–38	Management response (goals regress)

Table 15: Financial consolidation $V(t)$ trajectory. V reaches 0.00 at resolution epochs and exhibits the sawtooth pattern (convergence $\rightarrow$ cross-epoch shock $\rightarrow$ re-convergence) observed in Exp. 1, confirming domain-independent convergence behavior.

The temporal restatement demonstrates dual temporality in action: Alpha's restated Q1 revenue (EUR 126.6M vs original EUR 127.4M) is recorded with transaction time May 22, superseding the April 14 original for the same valid period (Q1 2025). The system sets `superseded_at` on original nodes. With goal-aware stale protection, goals persist across extraction cycles and are resolved at resolution rounds, producing the same sawtooth $V(t)$ pattern as Exp. 1: V drops to 0.00 at resolution epochs (24, 28–30, 36), then rises back to 0.25 as new documents introduce new goals.

Three properties distinguish this scenario: (1) contradiction detection respects valid-time overlap—Q1 figures contradict only other Q1 figures, not Q2; (2) the auditor's range-based observations (39–41% gross margin) are correctly classified as compatible, avoiding false contradictions; (3) the $V(t)$ sawtooth pattern ($0.25 \rightarrow 0.00 \rightarrow 0.25$) matches the M&A dynamics, confirming that convergence behavior—including full $V \rightarrow 0$ at resolution boundaries—is domain-independent. The scenario produced 45 convergence points across 38 epochs.

11 Discussion

11.1 What the Experiments Demonstrate

The experimental results provide evidence for four of the paper's central claims, while revealing important limitations in two others.

1. $V(t)$ tracks disagreement and converges to zero on resolution (confirmed). Across all conditions with contradictions, $V(t)$ increased reproducibly when contradicting documents entered the graph and *decreased to 0.0000* when resolutions were provided. In Exp. 1 ($c = 3$, resolution at rounds 5–7), V dropped from 0.250 to 0.0000 at resolution epochs, with goal score reaching 1.0000—full convergence. This pattern replicated across experiments: $V = 0$ was achieved in Exp. 1, 6, 7, 8, and the financial scenario. Cross-epoch document injection then re-introduces new goals, producing the sawtooth trajectory predicted by the perpetual lifecycle model. The Lyapunov function tracks genuine epistemic state bidirectionally: increasing with contradictions and new goals, decreasing to zero with resolutions.

2. Five-gate mechanism prevents false finality (confirmed). Exp. 3 demonstrates that the system reaches RESOLVED (score = 1.0) during the spike phase, then correctly exits RESOLVED when contradictions arrive. Gate A (monotonicity) and Gate B (evidence coverage) are the primary defenses: Gate B satisfaction drops from 100% to 58% when contradictions appear. The ablation (gates disabled) shows that without gates, the system is more permissive (RESOLVED 38% vs

33%, ESCALATED 0% vs 7%), confirming that gates provide an additional safety layer beyond threshold-based checking.

3. Governance mode affects coverage (confirmed). Exp. 5 demonstrates the coverage–autonomy trade-off across governance modes. YOLO processes all documents (23 decisions). MITL occupies an intermediate position with human-escalated review (40 decisions, all escalated). MASTER evaluates all proposals through the full reduction kernel, applying the three-stage pipeline (policy, lattice, mode routing) and only accepting proposals that pass both policy and lattice admissibility checks. Policy violations and lattice incomparability produce Reject, not Escalate. This maps directly to the lattice ordering Master $\leq$ Mitl $\leq$ Yolo in Definition 3 and aligns with SECP's finding that stricter evaluation reduces coverage.

4. Pressure-directed routing affects system dynamics (directional evidence). The pressure ablation shows a substantial difference in finality state distribution: pressure-enabled runs produce 94 convergence points with HITL dominating (84%), while pressure-disabled runs produce 27 points with ACTIVE dominating (63%). This suggests that pressure routing accelerates the system toward finality evaluation, though the single-run ablation design ($n = 1$) limits statistical confidence.

5. Multi-tier governance routing is exercised (confirmed). Exp. 7 demonstrates that the three governance modes produce observably different tier coverage: YOLO routes 97% of decisions through Tier 1, MITL escalates 100% to Tier 2 (human-in-the-loop), and MASTER rejects 30% of proposals at Tier 1 due to policy violations. Combined across modes, Tier 1 and Tier 2 are both exercised. This confirms that the lattice ordering Master $\leq$ Mitl $\leq$ Yolo produces structurally distinct governance behavior, not merely different labels on the same path.

6. Adversarial agents cause oscillating—not permanent—false finality (partially confirmed). Exp. 8 demonstrates that adversarial confidence inflation (single compromised agent) produces 3× more governance overrides than baseline (9 vs 3). With coordinated adversarial agents (facts + drift), the system oscillated between RESOLVED and HITL over 31 epochs (vs 18 for baseline), with 18 convergence snapshots reaching false RESOLVED. Cycle-based re-extraction flushed adversarial mutations within 1–2 cycles each time, preventing sustained false finality. The cooperative agent model (Assumption 5) is structurally necessary (adversarially induced RESOLVED windows exist, $V(t)$ monotonicity is violated), but the system has unexpected defense-in-depth: source re-extraction acts as a practical Byzantine defense.

7. Partial confluence holds for CRDT operations (confirmed). Exp. 9 demonstrates that the core semantic graph operations—confidence ratchet (max join), contradiction resolution (set-once), node upsert (content-matched)—are fully commutative. Claim stale marking is non-commutative (18/24 orderings diverge) but eventually consistent after complete re-extraction; goals and risks are protected from stale marking as accumulative types, narrowing the non-commutativity surface. The governance kernel is a pure deterministic function (80 evaluations, all identical). This validates the paper's claim of "partial confluence" and identifies claim stale marking as the specific operation preventing full commutativity.

11.2 What Remains Unproven

1. Full $V \to 0$ achieved within evidence epochs; cross-epoch oscillation persists. With goal-aware stale protection and progressive resolution (rounds 5–7), $V(t)$ reached 0.0000 at resolution epochs (epochs 9, 13, 17 in Exp. 1), with goal score reaching 1.0000 and all four dimensions at target. This is the first empirical confirmation that $V \to 0$ is achievable. However, cross-epoch document injection re-introduces new goals and contradictions, causing V to rise again (to 0.0078–0.0216). The overall trajectory is a *sawtooth*: convergence to 0 within each evidence epoch, followed by a cross-epoch shock when new documents arrive.

The intra/cross-epoch analysis reveals a subtlety: the regime boundary of Proposition 1 is finer than document injection. Facts-worker extraction can introduce new goals or contradictions *within* an evidence epoch—a non-governance operation that occurs alongside governance-approved transitions. The proposition's precondition ("governance-approved transitions") is formally correct, but the `context_seq` proxy includes non-governance operations. A finer-grained partition—separating governance-approved transitions from facts-extraction side effects—would be needed to test the proposition's intra-epoch guarantee directly.

Per-dimension analysis shows that cross-epoch oscillation is driven by the `goal_completion` dimension: new documents extract additional goals that are not yet resolved, raising V from 0.0000 to 0.008–0.022 before the next resolution cycle restores convergence. The other three dimensions (`claim_confidence`, `contradiction_resolution`, `risk_score_inverse`) remain at or near 1.00 throughout. The scalar metric collapses this per-dimension dynamic into apparent oscillation between two values, when the underlying behavior is a single dimension cycling between target and slightly below target.

The architecture partially addresses this limitation through the product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5), where the convergence rank $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ tracks each dimension independently. The reduction kernel's admissibility check (Definition 6) evaluates per-dimension regression rather than relying solely on the scalar aggregate, preventing transitions that improve the aggregate while regressing individual dimensions. However, the finality evaluator's ultimate decision still relies on the scalar goal score and gate predicates. A fully per-dimension finality criterion—requiring each dimension to independently satisfy its target—remains a direction for future work.

2. Multi-path governance (partially observed). In Exp. 4, all decision records used `governance_path = oversight_acceptDeterministic`—the oversight LLM always agreed with the deterministic policy result. Exp. 7 addressed this limitation by running each governance mode (YOLO, MITL, MASTER) as a separate sub-experiment with unified policy rules. MITL mode produced 100% Tier 2 escalation (all proposals required human approval), confirming that the routing architecture functions correctly. However, Tier 3 (full LLM governance via the oversight agent choosing `escalateToLLM`) was still not triggered: the oversight LLM consistently accepted the deterministic result even under medium and high drift. Triggering Tier 3 would require proposals where the deterministic and LLM-based evaluations genuinely disagree.

3. Scalability beyond $N = 100$ (not tested). Exp. 2 now covers $N \in \{10, 50, 100\}$. At $N = 100$ ($\rho = 0.3$), the system produced 45 convergence points and 34 decisions across 32 epochs—approximately double the $N = 10$–50 runs. This suggests scaling effects emerge around $N = 100$, but the paper's protocol specifies N up to 1000. Testing at $N = 500+$ is needed to identify the hypothesized scaling bottleneck in contradiction resolution.

11.3 Threats to Validity

Construct validity. The Lyapunov function $V(t)$ is a designed metric, not a ground-truth measure of knowledge quality. A decrease in V indicates movement toward configured thresholds, not necessarily toward correct knowledge. The experiments measure V's response to document injection, not its correlation with epistemic accuracy.

Internal validity. The document corpus is synthetic (though realistic). Contradictions are explicit and clearly stated, not subtle or ambiguous. The noisy corpus (available but not yet tested in batch mode) was designed to address this gap. LLM stochasticity is partially controlled by $n = 3$ repetitions, but larger n would increase confidence.

External validity. Experiments use two domains: M&A due diligence and financial consolidation. The financial scenario confirms that convergence dynamics ($V(t)$ spike-and-recovery) are domain-independent. Generalization to additional regulated domains (KYC/AML, clinical trials) is asserted by architectural argument (Appendix A) but not empirically validated beyond these two.

Statistical power. With $n = 3$ runs per condition, the standard deviations are informative (e.g., V std = 0.001–0.015) but formal hypothesis testing (e.g., paired t-tests) would require larger samples. The current results should be interpreted as reproducibility evidence, not as statistically powered claims.

11.4 Implications for Future Work

The experimental results identify four specific research directions, ordered by theoretical impact:

Per-dimension finality (beyond lattice admissibility). The product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) tracks each convergence dimension independently during governance evaluation, and the reduction kernel rejects transitions that regress any single dimension. However, the finality evaluator's ultimate decision still relies on a scalar goal score and gate predicates. Extending the finality gates to require per-dimension satisfaction would: (a) prevent declaring finality when individual dimensions have not reached their targets; (b) expose per-dimension convergence rates as first-class diagnostic signals; (c) provide the per-dimension α_d needed for meaningful ETA estimation; (d) align with SECP's non-scalar coordination, where Pareto filtering and constructive objections prevent the collapse of structured disagreement. This extension is straightforward within the lattice framework: finality requires $a_d \leq \epsilon_d$ for each dimension d in the convergence rank, rather than a single scalar threshold.

Autonomous goal resolution. The resolver agent cross-references evidence and produces resolves edges for contradictions. Goal-aware stale protection allows `goal_completion` to advance when goals are resolved by the experiment driver or resolver agent. The remaining gap is a dedicated goal-resolution agent that transitions goals to resolved status autonomously when supporting evidence reaches sufficient confidence—removing the dependency on the experiment driver's `injectResolution()` mechanism and closing the convergence loop end-to-end within a single evidence epoch.

Finer-grained epoch boundaries. The `context_seq` proxy for evidence epochs includes non-governance operations (facts extraction, drift detection) alongside governance-approved transitions. A finer partition—tracking the exact governance transition that modifies the graph—would enable direct testing of the proposition's intra-epoch guarantee.

Production-scale and multi-domain validation. Testing at $N > 500$ documents, with noisy/ambiguous corpora, and across multiple regulated domains (KYC/AML, clinical trials) would establish external validity.

12 Scope, Limitations, and Future Work

Following the principled scoping methodology of de la Chica Rodriguez and Vera Díaz [5], we state explicitly and without mitigation rhetoric what this work establishes and what it does not.

12.1 What the Paper Demonstrates

1. **Architectural feasibility.** Declarative governance over shared immutable context is implementable with current technology (Postgres, NATS, LLMs, OpenFGA, native Rust via napi-rs) and produces a functioning coordination system with 121 Rust unit tests, 306 TypeScript tests, and 7 convergence benchmarks.

2. **Formal convergence tracking with empirical $V \to 0$.** The Lyapunov disagreement function $V(t)$ with five independent gates prevents premature finality in all tested scenarios and reaches $V = 0.0000$ at resolution epochs—the first empirical confirmation that the convergence mechanism achieves its theoretical target. Proposition 1 provides a proof sketch for monotonic progress under CRDT constraints, now empirically confirmed within evidence epochs.

3. **Product lattice governance.** The product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) provides a well-founded ordering over governance decisions, and the Epoch Termination Theorem (Theorem 1) guarantees finite convergence within evidence epochs. The reduction kernel (Definition 8) implements this ordering as a deterministic three-stage pipeline in compiled Rust.

4. **Perpetual lifecycle.** The architecture supports re-opening finality on new evidence without information loss, producing a chain of signed certificates that models the perpetual nature of regulated knowledge.

5. **Three-tier governance resilience.** The system continues to make governance decisions without LLM availability, falling back to deterministic rule evaluation via the circuit-breaker mechanism.

6. **End-to-end validation across two domains.** The Project Horizon M&A scenario demonstrates 83% autonomous contradiction resolution across 5 contradictory documents in 12 convergence rounds. A second scenario—financial consolidation with bitemporal restatement—confirms that the same spike-and-recovery dynamics hold when dual temporality is structurally necessary, with valid-time-scoped contradiction detection and temporal supersession operating correctly across 42 epochs.

7. **Coverage–autonomy trade-off.** The three governance modes (YOLO, MITL, MASTER) provide distinct positions on the coverage–autonomy spectrum identified by [5]: YOLO maximizes coverage (analogous to scalar aggregation), MASTER maximizes evaluator autonomy

through the reduction kernel (policy violations and lattice incomparability produce rejections), and MITL provides an intermediate, auditable regime with human escalation.

12.2 What the Paper Does Not Demonstrate

1. **Statistical generality or significance.** Project Horizon is a single scenario with synthetic documents. No confidence intervals, hypothesis tests, or distributional claims are supported. Results are existence proofs, not performance estimates.

2. **Formal Byzantine fault tolerance.** All agents are assumed cooperative. Exp. 8 demonstrates that adversarial agents can cause oscillating false finality (18 RESOLVED snapshots across 31 epochs with coordinated collusion), but cycle-based re-extraction limits each adversarial window to 1–2 extraction cycles. This is a practical defense, not a formal Byzantine guarantee: the system provides no proven $n \geq 3f + 1$ tolerance. The monotonic confidence constraint limits downward manipulation but does not prevent upward inflation. Formal Byzantine-tolerant coordination [3, 10, 23] remains future work, though the empirical resilience is stronger than previously claimed.

3. **Scalability beyond proof of concept.** Validation covers two scenarios (5 and 8 documents, $\sim$50 claims each, 7 agents). Whether convergence dynamics, governance throughput, or the Lyapunov function's behavior change qualitatively at production scale (thousands of claims, hundreds of agents) is unknown.

4. **Machine-checked convergence proofs.** Proposition 1 is a proof sketch validated empirically. No formal verification tool or proof assistant has checked the convergence guarantee. An arbitrary sequence of approved transitions could, in principle, violate the proposition's preconditions in untested scenarios.

5. **Convergence under real LLM stochasticity.** All convergence benchmarks use synthetic trajectories. Whether the oscillation detection (Gate C) and trajectory quality score Q perform correctly under the inherent stochasticity of real LLM reasoning traces is untested.

6. **Multi-iteration protocol self-modification.** Governance rules are declarative but static within a deployment. The system does not implement governed self-modification of its own coordination rules, as demonstrated by SECP [5]. Whether the Lyapunov convergence mechanism can serve as a formal foundation for evaluating protocol modifications across multiple iterations is a promising but unvalidated direction.

7. **Multi-scope and hierarchical finality.** Cross-scope governance, parent–child finality dependencies, and inter-scope certificate chains are designed in the architecture but not validated experimentally.

8. **Human-in-the-loop effectiveness.** The HITL review mechanism is designed and implemented but not empirically evaluated with real human reviewers.

9. **Coverage–autonomy formalization.** While the governance lattice $\mathcal{L}$ (Definition 3) provides a formal ordering of governance strictness, and the reduction kernel implements distinct routing per level, no formal metric quantifies evaluator autonomy as a continuous variable. The lattice provides a qualitative ordering ($\textsc{Master} \leq \textsc{Mitl} \leq \textsc{Yolo}$); a quantitative measure (e.g., the probability that a single agent's objection blocks finality) and its systematic variation across governance configurations would provide a principled basis for mode selection.

Any claim beyond the narrow statement—"the architecture is implementable, produces auditable convergence in a controlled scenario, and provides formal safeguards against premature finality"—is unsupported by the evidence presented.

12.3 Implementation Limitations

Dual temporality utilization. The bitemporal schema is structurally complete (valid-time and transaction-time columns, as-of queries, temporal contradiction detection) and operationally validated in the financial consolidation scenario, where temporal restatements (same valid period, later transaction time) correctly trigger supersession and valid-time-scoped contradiction detection. However, systematic population of `valid_from`/`valid_to` from unstructured document metadata (e.g., "FY2024 revenue" → `valid_from: 2024-01-01, valid_to: 2025-01-01`) requires temporal NLP or structured metadata ingestion. In the M&A scenario, most nodes are created without explicit valid-time. The financial consolidation scenario demonstrates what becomes possible when temporal metadata is available.

Evidence schemas. The evidence schema mechanism (Gate B) is framework-ready but currently configured with empty required types and no staleness constraints. Domain-specific schemas (e.g., M&A due diligence requiring financial, legal, technical, and compliance evidence types with maximum age constraints) must be authored per deployment.

Obligation handlers. The obligation enforcement framework provides a registry-based architecture and execution pipeline, but no handlers are currently registered. Real deployments will need handlers for dual review, compliance notification, escalation to external systems, and audit-entry generation.

12.4 Scalability Limitations

Horizontal scaling. The current implementation operates on a single Postgres instance, a single NATS stream, and a single governance agent process. While the architecture (event-driven, scope-partitioned, stateless agents) is designed for horizontal scaling, no sharding strategy, multi-instance governance, or distributed CAS has been validated. Production deployments at enterprise scale will require these.

Graph scale. Validation at 50 claims with 30% logical overlap. Production scale (thousands of claims, hundreds of agents) requires semantic graph optimization (approximate nearest-neighbor search, event log sharding, compiled governance rules).

12.5 Theoretical Limitations

Convergence proof gap. Proposition 1 and Theorem 1 provide proof sketches, not machine-checked proofs. The proposition's preconditions (every approved transition satisfies monotonicity constraints and makes progress on at least one dimension) are enforced by the system's design—specifically, the reduction kernel's admissibility check rejects transitions that violate monotonicity—but are not formally verified. The Epoch Termination Theorem's well-foundedness argument depends on the finiteness of $\mathcal{L}$ and the bounded descent property of $\mathcal{A}$ within an evidence epoch; both hold by construction but have not been machine-checked. A rigorous proof would require formalizing the governance function, the CRDT semantics, the lattice ordering on $\mathcal{M}$, and the gate predicates in a proof assistant such as Coq or Lean.

Scalar finality criterion (addressed). The product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ tracks convergence per-dimension during governance evaluation (the reduction kernel rejects transitions that regress any single dimension). The finality evaluator now supports both scalar and vector finality modes.

When `per_dimension_finality.enabled` is true, the vector predicate $F^*(t)$ (Section 12.8) requires each dimension to independently satisfy its threshold with per-dimension gates $G_A^{(d)}$ and $G_C^{(d)}$. This addresses the scalar aggregation that SECP [5] criticizes: both governance decisions and finality decisions are now fully lattice-aware. The scalar path is preserved as a configuration-driven fallback for backward compatibility.

Static governance rules. The governance function (the reduction kernel $\mathcal{K}$, Definition 8) and the lattice ordering on $\mathcal{M}$ are fixed within a deployment. Unlike SECP's self-modifying coordination protocols, our system does not adapt its governance rules or lattice structure based on observed outcomes. Integrating governed protocol modification [5] with the product lattice framework is a natural extension: each protocol modification would be evaluated by its effect on the lattice point $(l, a) \in \mathcal{M}$, with the reduction kernel preventing modifications that would violate the well-founded descent property required by Theorem 1.

Coverage–autonomy trade-off. The coverage–autonomy trade-off identified by [5] manifests in the governance lattice $\mathcal{L}$ (Definition 3): the lattice ordering MASTER $\leq$ MITL $\leq$ YOLO provides a qualitative ranking of governance strictness, and the reduction kernel implements distinct acceptance/rejection/escalation routing per level. However, the trade-off is not yet formalized quantitatively. Defining evaluator autonomy as a measurable quantity (e.g., the probability that a single agent's objection blocks finality, parameterized by $\ell \in \mathcal{L}$) and mapping it against coverage across governance configurations would provide a principled basis for mode selection.

12.6 Formal Assumptions and Validation Status

The following assumptions underpin the paper's formal claims. They are stated explicitly to prevent over-interpretation of the theoretical results. A sixth assumption—non-compensability (A6)—was previously open and is now closed by the vector finality implementation (Section 12.8): the vector predicate $F^*(t)$ blocks finality when any required dimension fails its threshold, preventing scalar compensation artifacts.

1. **Discretization of $\mathcal{A}$.** The Epoch Termination Theorem (Theorem 1) requires $\mathcal{M}$ to be well-founded. $\mathcal{L}$ is finite (3 elements), but $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ is not well-founded under the standard real ordering. The proof assumes a discretization with minimum step $\epsilon > 0$, making $\mathcal{A}_\epsilon$ finite. This is an implementation-dependent assumption: if the convergence evaluator uses continuous-valued scores without rounding, the well-foundedness argument does not apply. The theorem should be understood as holding for the discrete implementation, not for the mathematical abstraction.

2. **Local confluence (partially validated).** The connection to rewriting theory invokes Newman's Lemma, which requires local confluence as a precondition. Exp. 9 tested this property with six sub-tests across 24 permutations and 80 kernel evaluations. Result: core CRDT operations (confidence ratchet, contradiction resolution, node upsert) are fully commutative—the confidence ratchet implements max join (commutative, associative, idempotent), and contradiction resolution is set-once. Claim stale marking is non-commutative: 18/24 orderings diverge because the last extraction determines which claims are marked irrelevant (goals and risks are protected from stale marking as accumulative types). However, all orderings converge to the same state after a complete re-extraction cycle (eventual consistency). The governance kernel is a pure deterministic function (same inputs produce identical outputs regardless of evaluation history). The paper's claim of "partial confluence" is validated: certified compatible transitions commute; full Church–Rosser confluence does not hold due to claim stale marking but is compensated by eventual consistency.

3. **Monotonic progress precondition (empirically confirmed).** Proposition 1 and its corollary require that "at least one convergence dimension makes strict progress" per cycle. This precondition is satisfied in Exp. 1: within evidence epochs where resolutions are provided, `goal_completion` advances and $V(t)$ reaches 0.0000. Goals and risks are protected from stale marking as accumulative node types, ensuring the resolution mechanism can advance them. The practical limitation is that progress depends on resolutions being provided (by the resolver agent or experiment driver) within each epoch; without resolutions, the system plateaus.

4. **Tier 2/3 governance routing (partially validated).** Exp. 7 demonstrated that the three governance modes produce distinct tier coverage: MITL mode escalated 100% of proposals to Tier 2 (human-in-the-loop), while YOLO and MASTER operated at Tier 1. Combined across modes, Tiers 1 and 2 are both exercised, and MASTER's rejection behavior (30% of proposals blocked by policy) confirms that mode-dependent routing is operational. Tier 3 (full LLM governance, where the oversight agent chooses `escalateToLLM`) was not triggered in any experiment: the oversight LLM consistently accepted the deterministic result. The system's behavior under genuine Tier 2/3 disagreement remains unknown.

5. **Cooperative agent model (partially validated).** All formal guarantees assume cooperative agents (Threat Model, Section 3). Exp. 8 tested this assumption with two adversarial scenarios: single-agent inflation and coordinated collusion. Result: the monotonic confidence ratchet defends against downward manipulation but not upward inflation (confirmed). However, cycle-based re-extraction from source documents provides an unintended defense-in-depth: adversarial mutations are flushed within 1–2 extraction cycles. Single-agent inflation was caught by the honest drift agent (3× more governance overrides, 9 vs 3 baseline). Coordinated collusion produced 18 RESOLVED convergence snapshots across 31 epochs (vs 18 epochs baseline), with $V(t)$ oscillating between 0 and 0.25; each false RESOLVED window lasted only 1–2 extraction cycles before re-extraction flushed the adversarial state. Formal guarantees do not hold under adversarial conditions, but cycle-based re-extraction limits the practical impact of each manipulation. The cooperative assumption is structurally necessary for the formal theory, but the system's practical resilience exceeds what the theory guarantees. Formal Byzantine fault tolerance [3, 10] remains unaddressed.

12.7 Future Work: Research Directions

1. **Multi-iteration convergence experiments.** Run repeated convergence cycles with incremental evidence injection to characterize $V(t)$ trajectory shapes, convergence time distributions, and gate activation patterns (Section 9).

2. **Scalability benchmarks.** Systematically vary graph size, contradiction density, and agent count to identify scaling bottlenecks and validate pressure-directed activation at scale.

3. **Formal Byzantine agent integration.** Exp. 8 demonstrated that cycle-based re-extraction provides practical resilience against adversarial agents (ephemeral false finality limited to 1–2 cycles). However, this is a structural property, not a formal guarantee. Extending to formal Byzantine fault tolerance would require: (a) formalizing the re-extraction defense as a consistency mechanism with provable bounds; (b) adopting confidence-weighted consensus [23] or adapting the SECP non-scalar coordination mechanism to handle adversarial module outputs; (c) proving that n-agent collusion ($n < f$ for a given f-tolerance) cannot achieve sustained false finality.

4. **Governed protocol evolution.** Integrate SECP-style bounded self-modification [5] with the product lattice framework, using the well-founded descent on $\mathcal{M}$ as the invariant-preservation mechanism: a protocol modification is admissible if it does not introduce governance transitions that would violate the descent property required by Theorem 1.

5. **Multi-scope hierarchical finality.** Validate cross-scope governance where a parent scope reaches finality only when all child scopes are resolved, with certificate chains reflecting the hierarchy.

6. **Formal verification.** Encode Proposition 1, Theorem 1, the product lattice ordering, admissibility classification, and gate predicates in a proof assistant (Coq or Lean) to obtain machine-checked guarantees for the convergence and termination arguments.

7. **Real LLM trajectory validation.** Replace synthetic benchmarks with recorded LLM reasoning traces to validate oscillation detection and trajectory quality mechanisms under real stochasticity.

8. **Human-in-the-loop evaluation.** Conduct user studies with domain experts to measure HITL review effectiveness, decision quality, and time-to-resolution.

12.8 Formal Hardening: Per-Dimension Finality (Implemented)

The formal hardening program aligns the finality semantics with the lattice semantics that governance already uses, and systematically validates the assumptions that underpin the paper's formal claims. This has been implemented as part of Issue #18.

Per-dimension finality (implemented). The scalar finality criterion has been replaced with a vector predicate that requires each dimension to independently satisfy its threshold:

$$F^*(t) = \bigwedge_d \left[e_d(t) \leq \epsilon_d \ \wedge \ G_A^{(d)} \ \wedge \ G_C^{(d)} \right] \ \wedge \ G_B \ \wedge \ G_D \ \wedge \ G_E$$

where $e_d(t) = \max(0,\, \tau_d - \mu_d(t))$ is the per-dimension gap, τ_d the per-dimension threshold, $G_A^{(d)}$ the per-dimension monotonicity gate, and $G_C^{(d)}$ the per-dimension trajectory quality gate. Global gates G_B (evidence), G_D (quiescence), G_E (content) remain scalar. The implementation includes veto dimensions (contradiction resolution) whose failure unconditionally blocks finality, compensation detection (logging when scalar would pass but vector blocks), and a configuration-driven fallback to scalar finality for backward compatibility. This addresses the non-scalar convergence criterion that SECP [5] identifies as essential.

Assumption matrix (documented). Seven assumptions (A1–A7) underpin the formal claims: well-founded descent of $\mathcal{A}$ (A1, empirically validated), kernel determinism (A2, proven: 141 Rust tests), monotone CRDT evolution (A3, proven: SQL guards and CRDT invariants), progress precondition (A4, empirically validated: Exp. 6), local confluence boundary (A5, partially validated: Exp. 9), non-compensability (A6, **implemented**: vector finality predicate with 20 Rust + 13 TypeScript tests), and tier completeness (A7, partially validated: Exp. 7). Assumption A6 was the critical open item and is now closed by the vector finality implementation.

Proof obligations (PO-2 closed). Four proof obligations were defined: (PO-1) epoch termination with per-dimension rank function $r(t) = \sum_d \max(0, \tau_d - \mu_d(t))$; (PO-2) per-dimension finality soundness; (PO-3) eventual consistency safety; (PO-4) routing completeness. PO-2 is proven by construction: the test `vector_finality_blocks_compensation` provides a concrete witness where

scalar score $= 0.94 \geq 0.92$ (passes) but contradiction resolution $= 0.80 < 0.95$ (vector blocks). The remaining obligations await extended experiments.

Extended experiment program. Five new experiments are planned: (E1) scalar vs vector finality on paired corpora, measuring false finality rate and blocked compensation cases; (E2) Tier-3 reachability via deterministic-vs-LLM disagreement corpus; (E3) discretization sweep over ϵ sensitivity; (E4) confluence boundary extension beyond Exp. 9; (E5) adversarial compensation attack comparing scalar vs vector finality defenses. E5 is the key falsification experiment for PO-2.

13 Conclusion

13.1 Summary of Contributions

This paper presents a paradigm for autonomous agent coordination based on declarative governance over shared immutable context. The core contributions are: (i) a Lyapunov disagreement function with five independent gates providing layered guarantees against premature finality; (ii) a bitemporal CRDT-inspired semantic graph with monotonicity invariants; (iii) a product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ with an Epoch Termination Theorem guaranteeing finite convergence via well-founded descent; (iv) a deterministic Rust reduction kernel ensuring auditable, replayable governance independent of LLM stochasticity; (v) perpetual finality with Ed25519-signed certificate chains; (vi) three-tier governance routing with LLM-free fallback; and (vii) validation on two domains—M&A due diligence and financial consolidation with bitemporal restatement—demonstrating $V(t) \to 0$ at resolution epochs and domain-independent sawtooth dynamics across evidence epochs, validated by 141 Rust and 319 TypeScript tests.

13.2 Positioning Relative to Self-Evolving Coordination

This work and the Self-Evolving Coordination Protocol (SECP) study [5] address complementary halves of a shared problem. Where SECP demonstrates that bounded, governed self-modification of coordination protocols is technically feasible in a single-shot design, we demonstrate that formal convergence tracking over shared state provides the multi-iteration foundation such mechanisms need for sustained deployment. Specifically:

- SECP's open question on multi-iteration dynamics is addressed by our Lyapunov function $V(t)$ and the Epoch Termination Theorem (Theorem 1), which together guarantee finite convergence within each evidence epoch and track progress over indefinite cycles with formal safeguards against oscillation and premature finality.
- SECP's open question on audit infrastructure is addressed by our bitemporal graph and policy-version-bound certificate chain, which provide the temporal auditability their architecture currently checks empirically.
- The scalar convergence limitation is now fully addressed. The product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ tracks convergence per-dimension during governance evaluation, and the vector finality predicate $F^*(t)$ (Section 12.8) extends this to the finality decision itself, requiring each dimension to independently satisfy its threshold with per-dimension gates. This implements the non-scalar convergence criterion that SECP's Pareto filtering and constructive-objection requirements identify as essential.

- Our open question on adversarial agents is partially addressed by Exp. 8, which demonstrates that cycle-based re-extraction provides practical (not formal) Byzantine resilience, and by SECP's non-compensable objection rights, which provide a complementary formal mechanism.

A synthesis—governed protocol evolution evaluated by Lyapunov convergence, with the product lattice providing per-dimension governance guarantees within each cycle and formal finality gates across cycles—would address the remaining limitations of both approaches. The proposed experimental protocol (Section 9) is designed to build toward this synthesis.

13.3 Interpretation Boundaries

The empirical claims are narrow and intentionally circumscribed (Sections 12 and 11). Results from $n = 3$ repeated runs demonstrate reproducibility of $V(t)$ response to contradictions and gate mechanism effectiveness, but are not statistically powered distributional claims. The convergence guarantee (Proposition 1) and the Epoch Termination Theorem (Theorem 1) hold formally and were empirically confirmed: $V(t) = 0.0000$ was achieved at resolution epochs in Exp. 1, 2, 3, 7, and 8, and across three distinct convergence cycles in the financial consolidation scenario. Five formal assumptions were experimentally validated: discretization and monotonic progress (Exp. 6), multi-tier governance routing (Exp. 7), cooperative agent model (Exp. 8), and local confluence (Exp. 9)—all partially validated, identifying specific boundaries (stale marking for confluence, oscillating false finality limited to 1–2 cycles per window for adversarial resilience, Tier 3 non-triggering). The financial consolidation scenario confirms that convergence dynamics are domain-independent. No production-scale validation ($N > 50$ documents) has been performed.

13.4 Final Assessment

As LLM agents become increasingly capable of reasoning, planning, and hypothesis generation, the need for governance-based coordination—rather than pipeline-based orchestration—will grow. Regulated enterprises cannot adopt autonomous agents without the ability to audit, explain, and formally bound their behavior continuously as conditions evolve. The product lattice $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ provides the formal foundation: governance strictness and convergence progress jointly tracked in a well-founded ordering guaranteeing finite stabilization, while the deterministic reduction kernel ensures every decision is auditable and independent of LLM stochasticity. Validation across two domains—M&A due diligence and financial consolidation with bitemporal restatement—confirms that the convergence mechanism is domain-independent. Achieving full robustness requires combining this state-level convergence approach with protocol-level adaptability [5]. Neither alone is sufficient; together, they define the research agenda for governed autonomous agent systems in regulated environments.

Acknowledgments

This work was supported by Deal ex Machina SAS. We thank Jean-Christophe Jardinier for critical review of earlier drafts and detailed feedback on the formal convergence arguments. We also thank the communities behind PostgreSQL, NATS, OpenFGA, napi-rs, DSPy, and Mastra for foundational infrastructure.

References

[1] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998. ISBN 978-0521779203.

[2] David Elliott Bell and Leonard J. LaPadula. Secure computer systems: Mathematical foundations. Technical Report MTR-2547, MITRE Corporation, 1973. Foundational lattice-based security model.

[3] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, 1999.

[4] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002. ISBN 978-0521784511.

[5] Jose Manuel de la Chica Rodriguez and Juan Manuel Vera Díaz. Self-evolving coordination protocol in multi-agent AI systems: An exploratory systems feasibility study, 2026.

[6] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi: 10.1145/360051.360056.

[7] Marco Dorigo, Eric Bonabeau, and Guy Theraulaz. Ant algorithms and stigmergy. *Future Generation Computer Systems*, 16(8):851–871, 2000. doi: 10.1016/S0167-739X(00)00042-X.

[8] Dexter Kozen. Results on the propositional μ-calculus. *Theoretical Computer Science*, 27(3): 333–354, 1983. doi: 10.1016/0304-3975(82)90125-6.

[9] Shadaj Laddad, Alvin Cheung, and Joseph M. Hellerstein. Keep CALM and CRDT on, 2022. Published in Proceedings of the VLDB Endowment, 2023.

[10] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982. doi: 10.1145/357172.357176.

[11] LangChain Team. LangGraph: Building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024.

[12] Aleksandr Mikhailovich Lyapunov. The general problem of the stability of motion. *Annals of the Faculty of Sciences of Toulouse*, 1892. Reprinted in English by International Journal of Control, 1992.

[13] Yihuan Mao, Yipeng Kang, Peilun Li, Ning Zhang, Wei Xu, and Chongjie Zhang. IBGP: Imperfect Byzantine generals problem for zero-shot robustness in communicative multi-agent systems, 2024.

[14] João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://github.com/joaomdmoura/crewAI`, 2024.

[15] Maxwell Herman Alexander Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942. doi: 10.2307/1968867.

[16] OASIS. eXtensible Access Control Markup Language (XACML) version 3.0. Technical report, OASIS Standard, January 2013. URL `http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html`.

[17] Reza Olfati-Saber and Richard M. Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on Automatic Control*, 49(9):1520–1533, 2004. doi: 10.1109/TAC.2004.834113.

[18] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google's consistent, global authorization system. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*, pages 33–46, 2019.

[19] Wei Ren, Randal W. Beard, and Ella M. Atkins. A survey of consensus problems in multi-agent coordination. In *Proceedings of the 2005 American Control Conference*, pages 1859–1864. IEEE, 2005.

[20] Chaoyi Ruan, Yiliang Wang, Ziji Shi, and Jialin Li. Reaching agreement among reasoning LLM agents, 2025.

[21] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000. ISBN 978-1558604360.

[22] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.

[23] Lifan Zheng, Jiawei Chen, Qinghong Yin, Jingyuan Zhang, Xinyi Zeng, and Yu Tian. Rethinking the reliability of multi-agent system: A perspective from Byzantine fault tolerance, 2025.

A Enterprise and Regulatory Fitness

Regulated environments—financial services, healthcare, defense, critical infrastructure—impose requirements that go beyond functional correctness. This appendix maps the architecture's capabilities to specific regulatory demands, showing that the mechanisms described above are not merely useful but structurally necessary for enterprise deployment.

A.1 Temporal Audit for Regulators

Regulatory frameworks frequently require point-in-time reconstruction: demonstrating what was known, when, and what decisions followed. SOX Section 404 requires that internal controls over financial reporting be documented as they existed at the reporting date. IFRS 9 (Expected Credit Loss) requires temporal evidence chains for impairment assessments. Basel III mandates historical risk factor traceability.

The bitemporal semantic graph (Section 4.2) enables these queries directly:

- **"What did the system know on audit date T'?"** As-of transaction-time query: filters to rows with `recorded_at` $\leq T'$ and (`superseded_at` $> T'$ or not superseded).
- **"What was believed true for reporting period T?"** As-of valid-time query: filters to rows with `valid_from` $\leq T$ and (`valid_to` $> T$ or open-ended).
- **"What did the system know at T' about facts valid at T?"** Combined bitemporal query: both filters applied simultaneously.

Unlike retrospective log analysis (grepping timestamps in application logs), these are first-class database queries with indexed support, returning structured graph state rather than unstructured log lines.

A.2 Cryptographic Non-Repudiation

When a scope reaches finality (Section 5.7), an Ed25519-signed JWS certificate is issued containing: the scope identifier, the finality decision, the timestamp, and content hashes of the governance and finality policy files in effect. This certificate is self-contained: an external party—auditor, counterparty, regulator—can verify the signature and inspect the payload without access to the running system.

The policy-version hashes are critical: they bind the finality decision to the exact rule set that produced it. If governance rules are amended between two finality decisions, the certificates will carry different policy hashes, enabling precise attribution of which decisions were made under which rules.

This addresses requirements in MiFID II (best execution record-keeping), SOX (management sign-off chains), and EMIR (trade repository reporting), where demonstrating *which rules governed a decision* is as important as demonstrating the decision itself.

A.3 Summary of Remaining Regulatory Mappings

Several additional enterprise concerns—policy change management (each `DecisionRecord` binds to a content-hashed `policy_version`), ongoing compliance lifecycle (the perpetual finality mechanism produces certificate chains for KYC/AML, IFRS 9, Basel III, and pharmacovigilance monitoring), evidence freshness (Gate B enforces per-domain staleness limits), separation of duties (OpenFGA + governance modes map to SOX/PCI-DSS segregation requirements), operational resilience (circuit breaker, message deduplication, graceful degradation), and governance agility (YAML rule changes are auditable and require no recompilation)—are addressed by mechanisms described in Sections 4.3–5.8 and are not re-derived here. These mappings are assertion-based: they follow from the architectural properties established above but have not been validated in production regulatory contexts.